Implementierung und Evaluation eines Webtools zur kollaborativen Erzeugung von Testdatensätzen für statistische Analysewerkzeuge

Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Science (B.Sc.)

vorgelegt an der

Universität Bremen

Fachbereich 3

von

Adam Cihasev

Matrikelnummer: 4510509

Erstgutachter: Prof. Dr. Rainer Koschke

Zweitgutachter: Dr. Karsten Sohr

20.05.2023

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation 1

1.2 Problemstellung 2

1.3 Ziele 5

2 Grundlagen **6**

2.1 REST-API 6

2.2 OAuth 2.0 7

2.3 React 8

2.4 CVE 9

3 Lösungsansatz **11**

4 Anforderungsanalyse **22**

4.1 Funktionale Anforderungen 22

4.1.1 Rollen 22

4.1.2 Anforderungen 23

4.1.3 Berechtigungsübersicht 25

4.2 Nicht-funktionale Anforderungen 26

4.3 Use-Case-Diagramm 27

5 Konzeption und Implementierung **28**

5.1 Zugriff auf Informationen 28

5.2 Architektur 32

5.2.1 Technologien 32

5.2.2 Aufbau 34

5.2.3 Datenmodell 35

5.3 Grafische Oberfläche 38

5.4 Entwicklungsansatz 40

6 Evaluation **42**

6.1 Methode 42

6.2 Auswahl von Probanden 43

6.3 Experimente 44

6.3.1 Experiment 1 44

6.3.2 Experiment 2 48

6.3.3 Experiment 3 53

6.4 Diskussion der Ergebnisse 57

6.4.1 Interpretation 57

6.4.2 Validität 58

6.4.3 Limitierungen 59

7 Fazit und Ausblick **60**

1 Einleitung

1.1 Motivation

Der Begriff „statische Analyse“ bezieht sich auf jedes Verfahren zur Bewertung von Quellcode ohne dessen Ausführung. Mit statischen Analysetools, die eine solche Bewertung automatisieren, lässt sich eine Vielzahl von „Was-wenn“-Szenarien effizient untersuchen. Die Verwendung dieser Tools kann vor allem die Sicherheit von Anwendungen und Systemen verbessern, indem sie frühzeitig auf potenzielle Schwachstellen hinweisen. Zum Beispiel tragen sie dazu bei, dass unsichere Funktionen oder Bibliotheken vermieden werden, dass eine ausreichende Eingabe- und Ausgabevalidierung durchgeführt wird und dass der Code den gängigen Sicherheitsstandards entspricht. Allerdings haben statische Analysetools den Nachteil, dass sie oft Fehlalarme bzw. *False-Positives* erzeugen, insbesondere wenn sie auf komplexe Codestrukturen oder ungewöhnliche Codierungspraktiken stoßen. Diese Fehlalarme können zu einer Überlastung der Entwickler führen, da sie extra Zeit aufwenden müssen, um zu prüfen, ob die Warnungen tatsächlich relevant sind. Umgekehrt, können die Tools auch echte Probleme übersehen, was dann zu schwerwiegenden Konsequenzen führen kann [1].

Angesichts der beschriebenen Problematik arbeitet die Universität Bremen derzeit an einem Projekt des BSI (Bundesamtes für Sicherheit in der Informationstechnik), bei dem die Anwendbarkeit des maschinellen Lernverfahrens für *SAST* (*Static Application Security Testing*) eruiert werden soll. Der Anlass hierfür sind Forschungen und Erfahrungen, die darauf hindeuten, dass ML-Verfahren Vorteile bei der automatisierten Mustererkennung bieten können. Dadurch sollen sich Daten schneller und genauer analysieren lassen als mit herkömmlichen Mustererkennungsmethoden wie statistischen Methoden oder regelbasierten Systemen. Ein wichtiger Bestandteil des Projekts ist unter anderem, die ständige Evaluation von existierenden SAST-Tools und ultimativ auch die eines eigens entwickelten Prototyps, der auf ML-Verfahren aufbaut. Grundsätzlich zielt das Projekt darauf ab, die Entwicklung von ML-basierten SAST-Tools zur automatisierten Erkennung neuer Fehlermuster zu unterstützen und voranzutreiben.

Die Entwicklung eines ML-basierten SAST-Tools beinhaltet das Training eines Modells mit einem überwachten Lernansatz. Dabei erhält das Modell verschiedene Code-Samples als Eingabe und die zugehörigen Labels, die angeben, ob das Sample als sicher oder unsicher eingestuft wird. Auf dieser Basis lernt das Modell, eine Beziehung zwischen der

Eingabe und den Labels zu erkennen und kann somit automatisch potenzielle Sicherheitslücken in Quellcodes identifizieren. Für die Erstellung eines optimierten Modells, das in der Lage ist, möglichst korrekte Diagnosen für neue, unbekannte Quellcodes zu liefern, ist ein hochqualitativer Testdatensatz gefragt. Ein solcher Datensatz ist auch für gängige SAST-Tools notwendig, um die Fehlerquote zu messen, die anschließend zur Evaluation des Tools verwendet wird.

1.2 Problemstellung

Die Frage, die sich somit bei der Evaluation von SAST-Tools und beim Training von ML-basierten SAST-Tools stellt, ist, wie ein qualitativ hochwertiger Datensatz gewonnen werden kann.

In erster Linie ist hier die Nutzung von frei zugänglichen Datenbanken zu nennen. Ein gutes Beispiel dafür ist die Juliet-Test-Suite. Sie bietet eine umfangreiche Sammlung von Testfällen für statische Code-Analysetools, die verschiedene Programmiersprachen und Schwachstellen abdecken. Das Problem ist jedoch, dass diese Testfälle synthetisch erzeugt sind und daher nicht komplex genug, um realistische Szenarien abzubilden. Es gibt zwar auch Sammlungen realistischer Testfälle, wie z.B. das SVCP4C-Dataset, aber der Umfang, die Vielfalt und die Aktualität sind für die angestrebten Ziele nicht ausreichend.

Als einzige zuverlässige Lösung bleibt die manuelle Erstellung der Datensätze. Dies erfordert zunächst eine Vielzahl von verwundbaren Quellcodes. Ein geeigneter Ort, um nach solchen Quellcodes in großen Mengen zu suchen, sind öffentlich zugängliche Software-Repositorys, die auf Versionskontrollplattformen wie z.B. GitHub oder GitLab betrieben werden. Der entsprechende Prozess wird auch als *Repository Mining* bezeichnet. Bei der Suche nach den Quellcodes wird die Historie des Repositorys betrachtet, die sich aus zahlreichen *Commits* zusammensetzt. Jeder dieser Commits enthält Änderungen, die von einem Entwickler am Softwareprojekt vorgenommen und im Repository verzeichnet wurden, sowie zusätzliche Metadaten, wie z.B. einen Kommentar des Entwicklers. Falls es sich um Quellcode-Änderungen an der Datenverschlüsselung, Autorisierung, Eingabevalidierung oder anderen sicherheitsrelevanten Funktionen handelt, dann liegt ein sicherheitsrelevanter Commit vor, der direkt oder indirekt zur Erhöhung der Softwaresicherheit beiträgt. Zu dieser Art von Commits gehören auch

Sicherheitsfixes, die bestimmte Schwachstellen in der Software beheben. Gerade solche Commits sind von Interesse, denn dadurch lassen sich sowohl verwundbare Quellcodes als auch ihre sichere Version, die für das ML-Training relevant ist, erhalten.

Die manuelle Suche nach den Sicherheitsfixes in einem Repository kann viel Zeit in Anspruch nehmen, weshalb man auf frei verfügbare Tools wie „git-vuln-finder“ zurückgreift, um die Suche zu automatisieren. Die Zuverlässigkeit dieser Tools ist jedoch begrenzt, da sie bei der Suche nur die Nachricht des Commits berücksichtigen, die oft sehr knapp oder wenig aussagekräftig formuliert ist. Demnach müssen die Resultate dieser Tools manuell unterschieden werden: tatsächliche Sicherheitsfixes und reguläre Commits. Dies ist allerdings aus mehreren Gründen keine einfache Aufgabe:

P1 - Fehlende Informationen

Um einen Commit auf einen Sicherheitsfix hin zu analysieren und ggf. verwundbare Quellcodes darin zu identifizieren, werden Informationen benötigt. Dazu gehören zunächst die grundlegenden Informationen, wie die Commit-Änderungen und Nachricht, sowie zusätzliche Informationen, die in der Nachricht referenziert werden und die zum Verständnis des Commits beitragen können. Solche Referenzen können z.B. eine Ticket-Nummer sein, die sich auf eine bestimmte Aufgabe oder ein Problem bezieht, eine URL, die auf einen Fehlerbericht, eine Diskussion oder eine andere relevante Ressource verweist, oder die ID eines Commits, auf dem der aktuelle Commit aufbaut. Daneben können auch Daten aus dem Kontext hilfreich sein. Alle diese Informationen sind über verschiedene Stellen verstreut, was es schwierig macht, relevante Fakten schnell zu überblicken. Die Informationen, die zusammen vorliegen, sind oft unübersichtlich angeordnet, während die anderen in externen Quellen nachgeschlagen werden müssen.

P2 - Fehlende Analyseumgebung

Die Analyse einer Vielzahl von Commits stellt eine noch größere Herausforderung dar. Neben dem manuellen Zusammentragen von Informationen für jeden Commit, treten weitere Probleme auf, wie z.B. Schwierigkeiten beim Zugriff auf die Commits oder beim Sicherstellen des Analysefortschritts. Inhaltliche Aspekte, wie z.B. irrelevante Änderungen in einigen Commits oder zu umfangreiche Informationsinhalte, erschweren die Analyse zusätzlich. Auch wenn verschiedene Techniken und Tools helfen können, bleibt der Aufwand hoch, da ihre effiziente Nutzung gute Kenntnisse voraussetzt und der ständige Wechsel zwischen ihnen sowohl spürbare Zeitkosten als auch kognitive Belastung verursachen kann.

P3 - Komplizierte Datensatzbildung

Je nach Anwendungsfall wird der gefundene Sicherheitsfix entweder direkt verarbeitet oder zusammen mit den Informationen für eine spätere Verarbeitung gesichert. Bei der Verarbeitung müssen zunächst die Quellcodes von anderen Dateien abgetrennt und anhand der Änderungen auf Relevanz geprüft werden. Sofern ein relevanter Quellcode vorliegt, kann ein sicheres Code-Sample erstellt werden. Beim Sampling-Prozess wird ein Ausschnitt des gepatchten Codes ausgewählt, von irrelevanten Zeilen (z.B. Kommentaren) bereinigt und entsprechend des Sicherheitsstatus und ggf. der Schwachstelle gelabelt. Für verwundbare Code-Samples müssen die betroffenen Quellcodes in der Projektversion des vorherigen Commits identifiziert und demselben Verfahren unterzogen werden. Der ganze Prozess spielt eine wichtige Rolle, da er sich unmittelbar auf die Qualität des Datensatzes auswirkt. Durch seine manuelle Durchführung steigt die Fehlerwahrscheinlichkeit, was wiederum die Qualität beeinträchtigen kann.

P4 - Einschränkungen bei Kollaboration

Selbst nach einer sorgfältigen Analyse ist es nicht immer ersichtlich, ob ein Commit einen Sicherheitsfix darstellt oder ob der darin enthaltene Quellcode verwundbar ist. In solchen Fällen wird ein Mehrheitsentscheidungsverfahren angewandt, bei dem der Commit von weiteren Experten begutachtet wird und anhand aller Begutachtungen eine Entscheidung bzgl. der Einordnung getroffen wird. Obwohl das Verfahren auch mit bestehenden Möglichkeiten, z.B. über eine einfache E-Mail-Korrespondenz, abgebildet werden kann, ist der organisatorische Mehraufwand immens, insbesondere bei größeren Softwareprojekten, deren Commits meist eine Vielzahl von Änderungen umfassen. Dieser Umstand erhöht auch das Risiko von Fehlern, die dann zu der falschen Entscheidung führen können.

1.3 Ziele

Wie bereits gezeigt, gibt es eine Reihe von Problemen, die es schwierig machen, die relevanten Quellcodes in Commits eines Repositorys zu identifizieren und für die Datensatzerstellung zu verarbeiten. Daher ist eines der Ziele dieser Arbeit die Implementierung einer Anwendung, die diese Prozesse vereinfacht. Die Anwendung soll eine geeignete Umgebung für die effiziente Analyse sowie Datensatzerstellung bieten, und kollaboratives Arbeiten unterstützen.

Weiterhin soll mithilfe der implementierten Anwendung eruiert werden, welche spezifischen Informationen und Anwendungseigenschaften die Commit-Analyse verbessern können und in welchem Ausmaß. Aus dieser Zielsetzung lassen sich die folgenden zwei Forschungsfragen ableiten:

- F1 - Welche Informationen können die Identifikation und Klassifikation von Sicherheitsfixes verbessern?
- F2 - Welche Eigenschaften und Funktionen muss eine Anwendung bereitstellen, um eine effiziente und benutzerfreundliche Entdeckung von Sicherheitsfixes sowie Datensatzerstellung zu ermöglichen?

Die Antworten auf diese Fragen sollen bei der Entwicklung von Methoden zur effizienten Erkennung von Sicherheitsfixes helfen und als Grundlage für weitere Forschung auf dem Gebiet dienen.

2 Grundlagen

In diesem Kapitel werden die wichtigsten Begriffe und Konzepte, die für das Verständnis der Arbeit relevant sind, eingeführt und erklärt. Insbesondere werden die Konzepte der Technologien behandelt, die das Fundament der zu erstellenden Anwendung bilden. Dieses Kapitel bietet somit einen Überblick über die wichtigsten Aspekte, die im Laufe der Arbeit thematisiert werden.

2.1 REST-API

Für die Anwendung wird eine *REST-API* definiert, um die Daten auf dem Server zu verwalten. Es handelt sich um eine Art von Web-API, die auf den Prinzipien des *REST* (Representational State Transfer)-Paradigmas basiert. Diese sind darauf ausgelegt, eine effiziente, skalierbare und flexible Art der Kommunikation zwischen verschiedenen Systemen über das Internet zu schaffen. Mit REST-APIs können Entwickler Daten und Funktionen von Anwendungen nutzen, ohne deren interne Struktur zu kennen oder sich um die Kompatibilität zwischen verschiedenen Anwendungen kümmern zu müssen [2].

Ein wichtiges Prinzip von REST ist die Trennung von *Client* und *Server*. Der Client ist die Anwendung, die die Anfragen an den Server stellt und die Antworten empfängt. Der Server ist die Anwendung, die die Ressourcen bereitstellt und auf Anfragen reagiert. Da der Client und der Server unabhängig voneinander sind, lassen sie sich beliebig modifizieren, ohne dass die Funktionalität des anderen beeinträchtigt wird [2].

Ein anderes wichtiges Prinzip in REST ist die *Zustandslosigkeit*. Das bedeutet, dass der Server keinen Zustand über die Interaktionen des Clients speichert. Jede Anfrage muss alle Informationen enthalten, die der Server benötigt, um sie zu verarbeiten. Dadurch entfällt die Notwendigkeit für den Server, Sitzungsdaten zu speichern und zu verwalten, was zu einer verbesserten Geschwindigkeit und Skalierbarkeit führt [2].

Ein zentraler Bestandteil von REST sind *Ressourcen*. Sie stellen Objekte oder Entitäten dar, die eine eindeutige ID bzw. einen *URI* (*Uniform Resource Identifier*) haben und über eine API angefordert werden können. Eine Ressource kann z.B. ein bestimmter ein Bild oder eine Sammlung von Dokumenten sein. Dabei ist zu beachten, dass Ressourcen nicht nur auf Daten beschränkt sind, sondern auch auf andere Arten von Ressourcen wie z.B. auf Prozesse oder Dienste, die von der API bereitgestellt werden [2].

Eine *Repräsentation* ist eine Darstellung eines Ressourcenzustands, die von einem Client abgerufen oder an einen Server gesendet werden kann. Neben dem Zustand kann sie auch Metadaten enthalten, wie z.B. Eigenschaften der Ressource, Beziehungen zu anderen Ressourcen und Aktionen, die auf die Ressource angewendet werden können. Die Repräsentation kann in verschiedenen Formaten vorliegen, wie z.B. JSON oder XML. Der Client und der Server können sich auf ein bestimmtes Format einigen, das für die Übertragung der Daten verwendet werden soll [2].

Im Falle einer REST-API werden die üblichen HTTP-Methoden (GET, POST, PUT, DELETE, etc.) verwendet, um Aktionen auf Ressourcen auszuführen. Die GET-Anfrage liest die Repräsentation einer Ressource aus, die POST-Anfrage erstellt eine neue Ressource, die PUT-Anfrage aktualisiert und die DELETE-Anfrage löscht eine bestehende Ressource. Um den Erfolg oder Misserfolg einer Anfrage zu signalisieren, werden HTTP-Statuscodes verwendet. Zum Beispiel gibt ein „200 OK“-Statuscode an, dass die Anforderung erfolgreich war, während ein „404 Not-Found“-Statuscode angibt, dass die angeforderte Ressource nicht gefunden wurde [3].

2.2 OAuth 2.0

Die REST-API der Webanwendung wird durch das Protokoll *OAuth 2.0 (Open Authorization)* gesichert. Dies ist ein standardisiertes Sicherheitsprotokoll für die Autorisierung, das ein Verfahren beschreibt, mit dem Drittanwendungen einen beschränkten Zugriff auf die Ressourcen eines Benutzers erhalten können. Dadurch können Benutzer mit Anwendungen teilen, ohne ihre Anmeldedaten preisgeben zu müssen [4].

Eines der wichtigsten Konzepte von OAuth 2.0 ist das Konzept der *Grant Types* (Autorisierungsarten), die festlegen, wie der Benutzer die Autorisierung erteilt. Es gibt mehrere Grant-Types, darunter Autorisierungscode, Passwort, Client-Credentials und der implizite Typ [4]. Für die anstehende Implementierung ist nur der implizite Grant-Typ von Interesse, daher wird dieser im Folgenden kurz erklärt.

Wenn sich ein Benutzer bei einer Anwendung anmeldet, die den impliziten Grant verwendet, wird er an den Autorisierungsserver weitergeleitet. Dort wird der Benutzer aufgefordert, seine Anmeldedaten einzugeben und die Anwendung für den Zugriff auf seine Ressourcen zu autorisieren. Sobald die Autorisierung erteilt ist, wird ein *Access-Token* an die Anwendung ausgestellt [4].

Ein Access-Token ist ein kurzlebiges Token, das den Zugriff auf geschützte Ressourcen ermöglicht. Es enthält Informationen darüber, wer die Anwendung autorisiert hat, auf welche Ressourcen die Anwendung zugreifen darf und wie lange das Token gültig ist. Access-Tokens werden normalerweise vom Autorisierungsserver ausgehändigt und vom Ressourcenserver verifiziert [4].

Ein Access-Token kann in verschiedenen Formaten vorliegen, darunter JWT (JSON Web Token), SWT (Simple Web Token), OAuth 2.0 Bearer Token und Opaque Token. Am häufigsten wird aber das JWT-Format verwendet, weil es eine sichere Kodierung und Übertragung von Informationen im Token ermöglicht. Ein JWT-Token besteht aus drei Teilen: einem Header, einem Payload und einer Signatur. Der Header enthält Informationen darüber, wie das Token kodiert wurde, der Payload enthält die eigentlichen Informationen (z.B. die ID des Benutzers, seine Rolle, etc.) und die Signatur dient zur Verifizierung der Integrität des Tokens [5].

Zusammen mit einem Access-Token kann vom Autorisierungsserver ein *Refresh-Token* ausgestellt werden, mit dem das Access-Token nach Ablauf automatisch erneuert werden kann, ohne dass der Benutzer seine Anmeldedaten erneut eingeben muss. Dadurch kann die Anwendung die Sitzung des Benutzers aufrechterhalten, ohne ihn ständig zu zwingen, sich erneut anzumelden. Refresh-Tokens sind in der Regel länger gültig als Access-Tokens und können manchmal auch vom Autorisierungsserver entzogen werden, um den Zugriff auf Ressourcen zu widerrufen [4].

2.3 React

Die grafische Benutzeroberfläche der Anwendung wird mit *React* umgesetzt. Dies ist eine JavaScript-Bibliothek, mit der sich benutzerfreundliche und interaktive Oberflächen erstellen lassen. Die Bibliothek ermöglicht es, UIs in kleine, wiederverwendbare Komponenten zu unterteilen, die unabhängig voneinander entwickelt und getestet werden können. Dies macht die Wartung und Erweiterung von Anwendungen wesentlich einfacher [6]. Der Anwendungsbereich von React liegt hauptsächlich bei *Single-Page-Anwendungen* (*SPAs*), bei denen alle Inhalte auf einer einzigen Seite geladen werden, ohne dass die Seite neu geladen werden muss, sowie bei *Progressive-Web-Anwendungen* (*PWA*), die fortschrittliche Webtechnologien nutzen, um eine der nativen App ähnliche Erfahrung zu bieten.

Ein wichtiger Bestandteil von React ist die Verwendung von *VDOM* (*Virtual DOM*). Es handelt sich um eine leichtgewichtige Kopie des tatsächlichen *DOMs* (*Document Object Models*), das von React intern erstellt und verwaltet wird. Das DOM selbst ist eine Programmierschnittstelle, die eine strukturierte Repräsentation eines Web-Dokuments bereitstellt und dessen dynamische Manipulation ermöglicht. Wenn sich der Zustand oder die Eigenschaften einer Komponente ändern, wird ein neues VDOM erstellt und mit dem aktuellen VDOM verglichen. React identifiziert die Unterschiede und aktualisiert nur die Teile im DOM, die sich tatsächlich geändert haben. Auf diese Weise kann die Anzahl der erforderlichen DOM-Änderungen minimiert und die Anwendungsleistung deutlich verbessert werden [7].

2.4 CVE

CVE (*Common Vulnerabilities and Exposures*) ist ein öffentliches Verzeichnis von bekanntgewordenen Schwachstellen in verschiedenen Software- und Hardwareprodukten. Das Verzeichnis wird von der gemeinnützigen Organisation MITRE Corporation gepflegt und dient als standardisiertes Referenzsystem für Schwachstellen. Jeder Eintrag erhält eine eindeutige Kennung, die sogenannte CVE-Nummer oder CVE-ID, mit der eine spezifische Schwachstelle eindeutig identifiziert werden kann. Durch die Verwendung von CVE-IDs können IT-Experten, Entwickler und andere Interessengruppen schnell und präzise auf Schwachstellen verweisen, ohne Missverständnisse aufgrund unterschiedlicher Namen oder Bezeichnungen der Schwachstelle zu riskieren [8].

Ein CVE-Eintrag enthält Informationen, die helfen können, Sicherheitsprobleme zu verstehen, zu bewerten und zu beheben. Im Folgenden werden sie näher erläutert:

1. *CVE-ID*: Eine eindeutige Kennung für die Schwachstelle, die in der Regel aus dem Präfix „CVE-“, gefolgt von der Jahreszahl der Entdeckung und einer fortlaufenden Nummer besteht (z.B. CVE-2022-1234).

2. *Beschreibung*: Eine kurze Zusammenfassung der Schwachstelle, einschließlich der Art und Weise, wie sie ausgenutzt werden kann.

3. *CVSS-Score (Common Vulnerability Scoring System)*: Das CVSS ist ein Bewertungssystem, das den Schweregrad der Schwachstelle anhand von verschiedenen Kriterien wie der Auswirkung auf die Vertraulichkeit, Integrität und Verfügbarkeit sowie dem Aufwand für die Ausnutzung der Schwachstelle berechnet. Der CVSS-Score reicht von 0 bis 10, wobei höhere Werte einen höheren Schweregrad anzeigen.

4. *Betroffene Produkte und Hersteller*: Eine Liste der Produkte und Hersteller, die von der Schwachstelle betroffen sind.

5. *Lösungen*: Links zu Patches, Updates oder anderen Lösungen, die von den Herstellern bereitgestellt wurden, um die Schwachstelle zu beheben.

6. *Weitere Informationen*: Zusätzliche Details zur Schwachstelle, einschließlich ihrer Ursache, möglicher Auswirkungen und Empfehlungen zur Behebung.

Die Angaben in einem CVE-Eintrag können sich zudem im Laufe der Zeit ändern, z.B. wenn neue Informationen über die Schwachstelle verfügbar werden oder wenn sich die Bewertung des Schweregrads aufgrund neuer Erkenntnisse ändert [9].

3 Lösungsansatz

Nach der Einführung in die Grundkonzepte ist es an der Zeit, sich mit der Implementierung der Anwendung zu befassen. Wie aus der Problemstellung hervorgeht, bestehen mehrere Probleme, die die Beschaffung von verwundbaren Quellcodes aus Repositorys erschweren und verlangsamen. In diesem Kapitel wird für jedes dieser Probleme ein Lösungsansatz erarbeitet, der dann in der Anwendung implementiert und später auf seine Wirksamkeit zur Beantwortung der Forschungsfragen getestet wird.

L1 - Bereitstellung von Informationen

Das Problem P1 besagt, dass sowohl für die Analyse eines Commits als auch für die Identifikation relevanter Quellcodes darin, Informationen benötigt werden, die derzeit nur schwer zu erreichen sind. Dieser Aufwand soll entfallen, indem die Anwendung den Informationsbezug und deren übersichtliche Zusammenstellung weitestgehend automatisiert. Welche Informationen für die Analyse relevant sein könnten, wird im Folgenden diskutiert.

Als Erstes soll die Commit-Nachricht angezeigt werden, da sie eine Art Dokumentation der durchgeführten Änderungen ist. Sie dient dazu, die Motivation des Autors und die Gründe für die Änderungen zu vermitteln. Bei einem Sicherheitsfix kann die Nachricht wertvolle Informationen über das behobene Sicherheitsproblem liefern, die sowohl für die Identifikation als auch für die Klassifikation relevant sind. Ein gutes Beispiel für eine solche Nachricht sieht wie folgt aus:

```
smtp: avoid risk of buffer overflow in strtol

If the incoming len 5, but the buffer does not have a termination
after 5 bytes, the strtol() call may keep reading through the line
buffer until is exceeds its boundary. Fix by ensuring that we are
using a bounded read with a temporary buffer on the stack.

Bug: https://curl.haxx.se/docs/CVE-2019-3823.html
Reported-by: Brian Carpenter (Geeknik Labs)
CVE-2019-3823
```

Listing 1: Commit-Nachricht eines Sicherheitsfixes aus dem *Curl*-Repository [10]

Die Nachricht in Listing 1 enthält am Beginn eine kurze Erklärung, welches Sicherheitsproblem behoben wird und wo, eine Beschreibung des Problems und der Lösung (Zeilen 3-6), sowie weiterführende Informationen und Quellen (Zeilen 8-11).

Von den weiterführenden Angaben sind CVE-Identifikatoren mit Bezug auf die geschlossene Sicherheitslücke von Interesse (siehe Zeilen 8 & 10, Listing 1). Die Commit-Nachricht soll nach solchen Identifikatoren durchsucht werden, um mit ihrer Hilfe Einzelheiten über die Sicherheitslücke zu ermitteln und anzuzeigen. Dazu gehören vor allem eine genauere Beschreibung, der Schweregrad und der Angriffsvektor. Anhand dieser Einzelheiten können Experten ein besseres Verständnis dafür entwickeln, welche Risiken die Schwachstelle für ein System darstellt, sie klassifizieren und entscheiden, ob der von ihr betroffene Code nützlich ist. Die Beschreibung kann auch bei der Identifikation von betroffenen Quellcodes helfen, indem sie die verwundbaren Funktionen oder Stellen im Code aufzeigt.

Während die CVE auf spezifische Sicherheitslücken in bestimmten Softwareversionen abzielt, beschäftigt sich die *CWE* (*Common Weakness Enumeration*) mit den zugrundeliegenden Ursachen bzw. Kategorien der Sicherheitslücken. Sie umfasst typische Schwachstellen von Software- und Hardwareprodukten, die durch verschiedene Faktoren wie z.B. fehlerhafte Programmierung oder mangelhafte Systemkonfigurationen verursacht werden können [11]. Die folgende Abbildung zeigt, wie ein CWE-Eintrag aussehen kann:

CWE-121: Stack-based Buffer Overflow

Weakness ID: 121
Abstraction: Variant
Structure: Simple

Description

A stack-based buffer overflow condition is a condition where the buffer being overwritten is allocated on the stack (i.e., is a local variable or, rarely, a parameter to a function).

Alternate Terms

Stack Overflow: "Stack Overflow" is often used to mean the same thing as stack-based buffer overflow, however it is also used on occasion to mean stack exhaustion, usually a result from an excessively recursive function call. Due to the ambiguity of the term, use of stack overflow to describe either circumstance is discouraged.

Relationships

Relevant to the view "Research Concepts" (CWE-1000)

Nature	Type	ID	Name
ChildOf	B	787	Out-of-bounds Write
ChildOf	B	788	Access of Memory Location After End of Buffer

Abbildung 1: Beginn eines CWE-Eintrags, der einen Stack-basierten Pufferüberlauf beschreibt [12]

In CVE-Einträgen werden in der Regel ein oder mehrere CWE-Einträge referenziert. Somit könnten relevante Informationen aus den CWE-Einträgen bereitgestellt werden, um die Analyse zu unterstützen. Entscheidend ist dabei der CWE-Name, da er eine unmittelbare Einordnung der Sicherheitslücke bewirken kann.

Obwohl CVE-Einträge ein wichtiges Element zur Identifikation von Sicherheitsfixes sein können, werden sie in der Nachricht nicht immer erwähnt. Dagegen wird häufiger auf gemeldete Probleme bzw. *Issues* verwiesen, die durch den Commit beseitigt wurden. Dazu werden Ausdrücke, wie z.B. „closes #xxx“ oder „fixes #xxx“ verwendet, wobei die Issue-Nummer anstelle der „x“-Zeichen angegeben wird [13]. Diese Referenzen könnten als geeigneter Ersatz oder auch als Ergänzung dienen, indem die entsprechende Problembeschreibung aus dem jeweiligen Issue-Tracker angezeigt wird.

Des Weiteren sollen die im Commit vorgenommenen Änderungen angezeigt werden, indem für jede betroffene Datei die Unterschiede zwischen der alten und der neuen Version ausgegeben werden. Die Änderungen sind hauptsächlich für die Identifikation der betroffenen Quelldateien notwendig, können aber auch helfen, den Commit trotz einer informationsarmen Nachricht zu verstehen. Allein dadurch kann Code identifiziert werden, der z.B. für Pufferüberläufe oder Codeinjektionen verantwortlich ist, da diese Sicherheitslücken und ihre Korrekturen ein ausgeprägtes Muster haben. Zur verbesserten Erkennung von kritischen Änderungen und deren Auswirkungen soll eine abschnittsweise Ausgabe des restlichen Codes, der nicht Teil des eigentlichen Commits ist, ermöglicht werden.

Außerdem sollen Metadaten zur jeweiligen Commit-Datei angezeigt werden, was durchaus üblich ist. Der Dateipfad kann helfen, irrelevante Dateien (z.B. Testdateien) schnell zu erkennen und zu überspringen. Die Dateierweiterung kann Aufschluss über die verwendete Programmiersprache geben und damit einen Hinweis darauf, was für Sicherheitslücken der Code enthalten könnte.

Ein Commit kann aber auch nur ein Teil einer komplexeren Fehlerbehebung sein. Aus diesem Grund soll eine gewisse Anzahl vorheriger und nachfolgender Commits als Überblick präsentiert werden, um Experten einen Einblick in den Kontext zu geben, in dem die aktuellen Änderungen gemacht wurden. Dadurch können sie Rückschlüsse auf die Art des Commits ziehen und möglicherweise weitere Sicherheitsfixes entdecken.

Die Gesamtheit dieser Informationen kann ein vollständiges Bild davon vermitteln, was ein Commit bewirkt hat. Bei der Evaluation der Anwendung sollen sie zur Beantwortung der Forschungsfrage F1 herangezogen werden, indem sie von Experten auf ihre Nützlichkeit bei der Analyse von Commits geprüft werden.

L2 - Bereitstellung einer Analyseumgebung

Aus Problem P2 geht hervor, dass die Analyse einer großen Anzahl von Commits durch verschiedene technische und inhaltliche Probleme erschwert wird. Die Anwendung soll daher eine Umgebung bereitstellen, die spezielle Eigenschaften und Funktionen implementiert, um einen Großteil der Probleme zu beseitigen und somit die effiziente Analyse zu ermöglichen.

Als Erstes soll ein einfacher und schneller Zugang zu den Commits geschaffen werden. Zu diesem Zweck soll die Anwendung als Webanwendung entwickelt werden, die Mehrbenutzerfähigkeit unterstützt und Benutzerkontenverwaltung implementiert. Dadurch können mehrere Benutzer die Anwendung gleichzeitig nutzen, wobei jedem Benutzer ein isolierter Arbeitsbereich zum Speichern und Analysieren von Commits zur Verfügung steht. Der Vorteil einer solchen Anwendung ist vor allem, dass ein Benutzer jederzeit und überall auf seine Commits zugreifen und die Analyse fortsetzen kann, sofern eine Internetverbindung und ein Webbrowser vorhanden sind.

Um ein Repository bequem nach Sicherheitsfixes zu durchsuchen, soll die Möglichkeit zur Erstellung eines Projekts eingeführt werden. Dabei handelt es sich um eine Art Container, in dem Artefakte und Aktivitäten gesammelt werden, die bei der Zusammenstellung von datensatzrelevanten Quellcodes aus Commits eines Repositorys hilfreich sind. Artefakte sind Elemente, die im Rahmen des Projekts erzeugt oder verwendet werden, wie z.B. Commits und die zugehörigen Informationen. Aktivitäten hingegen beziehen sich auf bestimmte Handlungen oder Tätigkeiten, die im Rahmen des Projekts ausgeführt werden, wie z.B. die Bewertung von Commits (siehe Ansatz L3). Bei der Erstellung soll der Benutzer zur Angabe eines Repository-URLs aufgefordert werden. Das Repository soll dann mit einem bestehenden Tool auf potenzielle Sicherheitsfixes gescannt, und die Resultate im Projekt gesichert werden. Auf diese Weise wird eine Basis geschaffen, die weitestgehend frei von regulären Commits ist und sich daher gut als Ausgangspunkt für die Suche eignet.

Um möglichst schnell an Sicherheitsfixes zu kommen, sollen in einem Projekt verschiedene Filterfunktionen bereitgestellt werden. Eine dieser Funktionen ist für den Benutzer transparent und stellt sicher, dass nur die Commits angezeigt werden, deren Dateien geändert oder neu hinzugefügt wurden. Zudem sorgt sie dafür, dass die Änderungen nur für solche Dateien eingeblendet werden. Das Ziel ist dabei, umbenannte,

verschobene und gelöschte Dateien herauszufiltern, da diese in der Regel irrelevant sind und die Analyse verkomplizieren. Eine andere Funktion ermöglicht Commits und zugehörigen Dateien anhand bestimmter Dateierweiterungen zu selektieren. Dadurch können Dateien von der Analyse ausgeschlossen werden, die nicht der gewünschten Programmiersprache entsprechen, wie z.B. Konfigurationsdateien oder Bash-Skripte. Die letzte Filterfunktion ermöglicht das Filtern von Commits nach sicherheitsspezifischen Begriffen in ihren Nachrichten. Dadurch können Sicherheitsfixes schneller gefunden werden. Um eine erneute Einstellung der Filterparameter zu vermeiden, sollen sie persistiert und automatisch geladen werden, wenn die Sitzung fortgeführt wird.

Weiterhin sollen häufig genutzte Funktionen mit Hotkeys verknüpft werden, um ihre Ausführung und damit die Bedienung der Anwendung zu beschleunigen. Sofern eine Funktion an einen Hotkey gebunden ist, kann der Benutzer schnell und einfach darauf zugreifen, ohne den Mauszeiger bewegen oder nach dem entsprechenden Button suchen zu müssen. Diese Eigenschaft ist vor allem dann nützlich, wenn der Benutzer mit einem Touchpad arbeitet, da dieses oft langsamer und umständlicher zu bedienen ist.

Die Informationen zu einem Commit sollen in einem separaten *View* (dt. „Ansicht“) dargestellt werden. Das View soll zur effizienten Nutzung des Bildschirms nach dem *Tiling*-Prinzip in mehrere aneinanderliegende Fenster unterteilt sein. Jedes Fenster soll eine wichtige Information enthalten, um die Suche danach zu ersparen. Weniger relevante Informationen sollen je nach Bedarf mit Hotkeys ein- und ausgeblendet werden können. Die Größe jedes Fensters soll primär nach der Verfügbarkeit der Information ausgerichtet sein, aber auch ihre Relevanz berücksichtigen. Dadurch wird sichergestellt, dass oft nicht verfügbare Informationen keinen unnötigen Platz einnehmen und dass wichtige Informationen immer ausreichend Platz haben. In diesem Fall wird mehr Platz für die Änderungen eingeräumt, wobei diese pro Datei angezeigt werden sollen, um die Übersichtlichkeit nicht zu gefährden. Die Fenster im View sollen zudem scrollbar sein, sodass der Benutzer jederzeit schnell auf den nicht sichtbaren Teil zugreifen kann. Mithilfe von Hotkeys soll er zwischen den Fenstern wechseln können, indem er sie einzeln durchläuft oder direkt zu einem der Fenster springt. Eine Skizze des Views ist in Abbildung 2 zu sehen.

Im Commit-View sollen lange Texte, wie z.B. eine ausführliche Nachricht, zusätzlich in einem kurzen und prägnanten Satz wiedergegeben werden. Dies kann die Identifikation relevanter Commits verbessern, indem es dem Benutzer ermöglicht, entscheidende

Aspekte schneller zu erkennen, ohne den gesamten Text lesen zu müssen. Eine Alternative könnte darin bestehen, sicherheitsspezifische Begriffe im Text zu identifizieren und farblich zu markieren oder als Liste von Tags auszugeben.

Für die bequeme Analyse der Änderungen sollen im Commit-View zwei Darstellungsformate, nämlich *Split-View* und *Unified-View*, angeboten werden. Diese stammen aus dem Bereich der Softwareentwicklung und beziehen sich auf die Darstellung von Änderungen zwischen zwei verschiedenen Dateiversionen. Split-View zeigt die beiden Versionen der Datei nebeneinander an und hebt die Unterschiede farblich hervor. Diese Darstellung kann besonders nützlich sein, wenn es viele Änderungen in einer Datei gibt oder wenn die Änderungen an verschiedenen Stellen in der Datei gemacht wurden. Durch die seitliche Anordnung der beiden Dateiversionen kann es einfacher sein, die Änderungen visuell zu überprüfen [14]. Im Gegensatz dazu zeigt Unified-View die beiden Dateiversionen übereinander an und zeigt die Unterschiede als zusammenhängende Textblöcke an. Diese Darstellung kann sich besser eignen, wenn es nur wenige Änderungen in der Datei gibt oder wenn die Änderungen an aufeinanderfolgenden Stellen in der Datei gemacht wurden. Die Formate sollen mit *Syntax-Highlighting* erweitert werden, um die Quelldateien lesbarer und verständlicher zu machen. Dabei werden die Sprachkonstrukte unterschiedlich eingefärbt, um sie optisch voneinander zu trennen. Zusätzlich sollen an den Stellen, wo Unterschiede beginnen oder enden, Buttons platziert werden, um den angrenzenden Inhalt einblenden zu können.

Der Platz im Änderungsfenster reicht nicht aus, um zwei Dateien nebeneinander in voller Breite anzuzeigen. Auch das Einfügen von Zeilenumbrüchen ist in diesem Fall keine Lösung, da sie bei regelmäßigem Auftreten den Lesefluss stören können. Daher soll das horizontale Scrollen bei den einzelnen Dateien aktiviert werden, um den Inhalt, insbesondere den Quellcode, in seiner ursprünglichen Form lesen zu können und damit einen besseren Überblick über die Zusammenhänge zu erhalten. Um jedoch das lästige Scrollen bei einer Vielzahl langer Zeilen zu vermeiden, soll das Fenster im Vollbildmodus ausgeklappt werden können.

Die Commit-Views sollen in einem *Carousel* abgebildet werden. Dies ist ein Design-Element, das die Navigation durch mehrere horizontal angeordnete Inhalte ermöglicht [15]. Um die Navigation effizienter zu gestalten, soll die Verwendung von Hotkeys vorgesehen werden. Eine solche Konstruktion ermöglicht es, schnell zwischen dem

aktuellen und dem nächsten bzw. vorherigen Commit-View zu wechseln, was gerade bei der Analyse zahlreicher Commits besonders hilfreich sein kann.

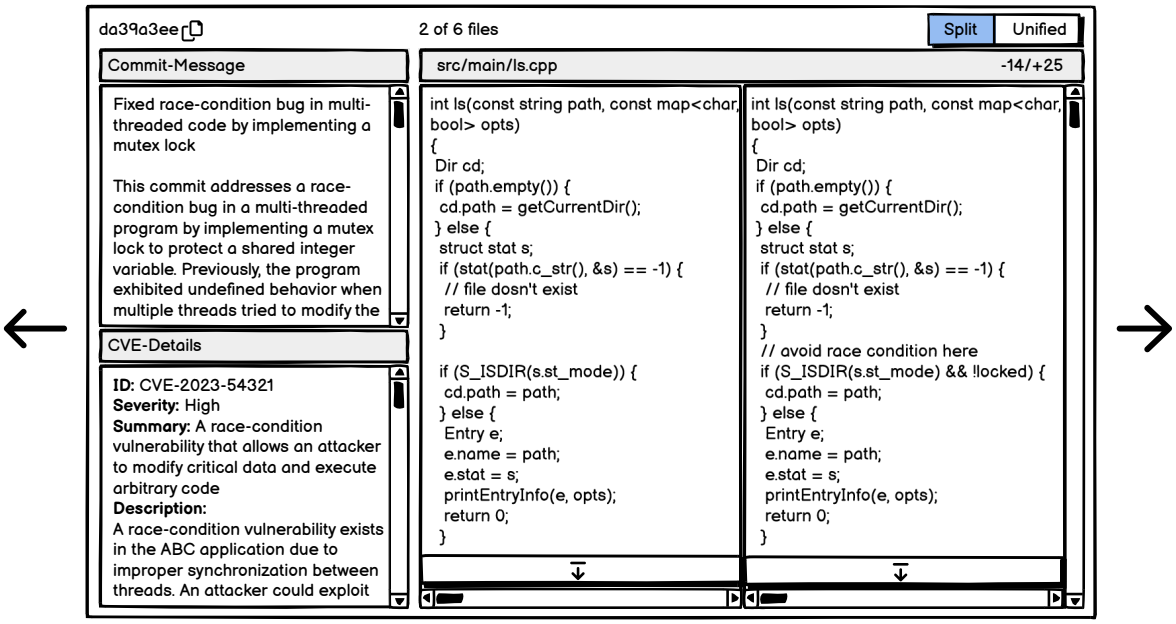

Abbildung 2: Layout des Commit-Views

Um die Forschungsfrage F2 zu beantworten, sollen im Rahmen von Usability-Tests die Funktionalität, Benutzerfreundlichkeit und Leistung der umgesetzten Anwendung überprüft werden. Die Tests sollen dabei von Experten durchgeführt werden, die Feedback zur Benutzerfreundlichkeit geben und Verbesserungspotenziale aufdecken. Anhand der beobachteten Probleme sollen die fehlenden Funktionen, die zukünftig in die Anwendung integriert werden können, ermittelt werden.

L3 - Möglichkeit zur effizienten Datensatzbildung

Das Problem P3 beschreibt den komplizierten Prozess, der zur Erstellung eines Datensatzes vollzogen werden muss. Mit der Anwendung soll der Prozess vereinfacht werden, indem die Quellcodes während der Analyse als sicher oder unsicher gekennzeichnet und schließlich in ein Format exportiert werden können, das als Grundlage für die effiziente Datensatzerstellung dient.

Für die Einstufung soll ein einfaches Bewertungssystem mit drei Stufen umgesetzt werden: „verwundbar“, „neutral“ und „nicht verwundbar“. Die Stufe „neutral“ bedeutet dabei, dass es momentan noch Unsicherheiten gibt. Die Bewertung soll über Commit-Views und für einzelne Dateien in einem Commit erfolgen, da eine Bewertung des

gesamten Commits aufgrund der möglichen Präsenz von sowohl relevanten als auch irrelevanten Dateien ungenau wäre. Zudem soll eine abgegebene Bewertung jederzeit korrigiert werden können, für den Fall, dass eine erneute Analyse ein anderes Ergebnis zeigt.

Nach der Definition einer Methode zur Einstufung von Commit-Dateien stellt sich die Frage, wie die eigentliche Datensatzerstellung automatisiert werden soll. Wie bereits in P3 erwähnt, werden die Datensätze aus gelabelten Code-Samples zusammengestellt, die jeweils ein sicheres bzw. unsicheres Quellcodefragment enthalten. Das Problem besteht darin, dass es keine einheitliche Verfahrensweise für die Auswahl solcher Fragmente gibt. In der Regel wird der Quellcode von einem Experten betrachtet, der anhand der gestellten Anforderungen und unter Berücksichtigung verschiedener Faktoren, wie z.B. der Relevanz der betroffenen Zeilen oder dem umgebenden Kontext, eine Auswahl trifft. Aus diesem Grund kann die Anwendung keine vollständigen Datensätze anbieten, dafür aber alle notwendigen Informationen, um sie effizient zu erstellen.

Sofern eine oder mehrere Commit-Dateien bewertet wurden, sollen Informationen darüber exportiert werden können. Die Exportdatei soll im *JSON*-Format (*JavaScript Object Notation*) vorliegen, einem leichtgewichtigen Datenformat zum Speichern und Übertragen strukturierter Daten [16]. Das Format wird von den meisten Programmiersprachen optimal unterstützt, sodass Informationen aus der Datei schnell geparst und verarbeitet werden können. Darüber hinaus ermöglicht die menschenlesbare und unkomplizierte Syntax eine direkte Überprüfung der Informationen. Die inhaltliche Struktur der Exportdatei sieht wie folgt aus:

```
1  {
2     "repository":"github.com/curl/curl",
3     "commits":[
4        {
5           "parent_hash":"dbd74baf781e33e95071a729a81c91a972eff0b7",
6           "commit_hash":"55807e6c056f27846d70cec70ee6ac3f0e5b3bbe",
7           "files":[
8              {
9                 "old_filepath":"lib/cfilters.c",
10                "new_filepath":"lib/cfilters.c",
11                "removed":[
12                   "137",
13                   "156",
14                   "430",
```

```
15                      "448"
16                  ],
17                  "added":[
18                      "262-273",
19                      "397-411"
20                  ],
21                  "votes":{
22                      "negative":0,
23                      "neutral":0,
24                      "positive":1
25                  }
26              }
27          ]
28      }
29  ]
30 }
```

Listing 2: Beispiel für Daten in der Exportdatei

Die Exportdatei soll einen Repository-URL (Zeile 2) sowie Informationen zu den bewerteten Commits und Dateien bereitstellen. Die Commit-Informationen enthalten nur Referenzen zur Identifikation des jeweiligen Commits, nämlich den Hashwert des vorherigen Commits für den unsicheren Quellcode und den Hashwert des aktuellen Commits für den sicheren Quellcode (Zeilen 5-6). Die Datei-Informationen bestehen aus dem alten und dem neuen Dateipfad (Zeilen 9-10), um die Datei im Falle einer Umbenennung oder Verschiebung aufzufinden, den Nummern der geänderten Zeilen (Zeilen 11-20), um bei Bedarf schnell darauf zugreifen zu können, und der Bewertung selbst (Zeilen 21-25). Die Bewertung soll dabei als ein Objekt erfasst werden, um sowohl Einzelbewertungen als auch Bewertungen von mehreren Personen im Rahmen einer Zusammenarbeit (siehe Ansatz L4) abbilden zu können.

Der Grund für die Angabe aller Bewertungen und nicht nur der vorherrschenden ist, dass es verschiedene Anwendungsfälle gibt, die unterschiedliche Anforderungen an Quelldateien stellen. Zum Beispiel können einfache, unsichere Datensätze auch solche Quellcodes gebrauchen, die zwar mehrheitlich als sicher eingestuft wurden, aber dennoch eine hohe Anzahl von Gegenstimmen erhielten, insbesondere wenn ein Mangel an unsicheren Quellcodes besteht. Umgekehrt, können für hochwertige Datensätze Quellcodes mit auch nur einer Gegenstimme unzulässig sein. Abgesehen von dieser

Tatsache kann ein Quellcode unter bestimmten Umständen nicht ohne Weiteres eingeordnet werden, z.B. wenn ein Gleichstand bei den Bewertungen vorliegt. Daher sollen die Bewertungsdaten dem Anwender zur Verfügung gestellt werden, sodass er anhand dieser Daten selbst entscheiden kann, wie die Dateien eingeordnet werden sollen.

Es fällt zudem auf, dass in der Exportdatei alle bewerteten Dateien verzeichnet werden, und nicht nur diejenigen, die als „verwundbar“ eingestuft wurden. Wie aus der Problemstellung bekannt ist, sind für das Training und die Evaluation auch sichere Quellcodes gefragt. Obwohl der Eindruck entstehen könnte, dass diese leichter zu beschaffen sind, gibt es keine Garantie dafür, dass es sich dabei wirklich um sichere Quellcodes handelt. Dies ist auf das *Rice*-Theorem zurückzuführen, wonach es keine algorithmische Möglichkeit gibt, eine nicht-triviale Eigenschaft von Turing-mächtigen Programmiersprachen zu entscheiden [17]. Wenn es um die Sicherheit von Quellcode geht, ist es daher unmöglich zu überprüfen, ob ein Quellcode sicher ist, da die Sicherheit eine nicht-triviale Eigenschaft ist, die auf einige, aber nicht auf alle Programme zutrifft. Somit kann ein als sicher geltender Quellcode neue Defekte erzeugen, die einfach noch nicht entdeckt wurden. Auch in diesem Fall erhält eine Mehrheitsentscheidung die Zuverlässigkeit, da sie verschiedene Perspektiven, Fähigkeiten und Kenntnisse zusammenführt, die die Wahrscheinlichkeit einer korrekten Einordnung erhöhen. Eine solche Mehrheitsentscheidung wird durch eine überwiegende Anzahl von „nicht verwundbar“-Stimmen abgebildet, was auch der Grund für die Aufzeichnung der entsprechenden Dateien ist.

L4 - Möglichkeit zur aktiven Kollaboration

Dem Problem P3 zufolge existiert derzeit keine benutzerfreundliche und effiziente Lösung, die es Experten ermöglicht, Sicherheitsfixes und die relevanten Quellcodes darin, gemeinsam zu identifizieren. Eine solche Lösung soll daher durch die Anwendung bereitgestellt werden. Wie in L2 beschrieben, soll die Anwendung als Webanwendung implementiert, die von mehreren Benutzern gleichzeitig über das Internet verwendet werden kann. Da alle Benutzer mit demselben Backend interagieren, lässt sich der Datenaustausch zwischen ihnen einfach realisieren. Es müssen nur noch geeignete Mechanismen definiert werden, um die einwandfreie Zusammenarbeit zu ermöglichen.

Nach L2 soll es die Möglichkeit geben, Projekte zu erstellen. Diese Projekte werden auch als Plattform für die Zusammenarbeit dienen. Dazu soll eine rollenbasierte Zugriffskontrolle ($RBAC$) umgesetzt werden, um eine sichere und kontrollierte Freigabe

des Zugangs zu einem Projekt zu ermöglichen. Dabei werden zwei Rollen definiert: Der Projektinhaber, der alle Zugriffsrechte auf das Projekt besitzt, und das Projektmitglied, das Commits im Projekt einsehen und bewerten kann. Zusätzlich soll ein Einladungssystem implementiert werden, das den Projektinhabern ermöglicht, Einladungen zur Teilnahme an andere Benutzer zu verschicken. Nachdem ein Benutzer eine solche Einladung akzeptiert hat, soll er automatisch Mitglied des jeweiligen Projekts werden und aktiv daran mitwirken können. Auch wenn P3 eine aktive Zusammenarbeit nicht verlangt, ist es das Ziel, die Qualität des gesamten Datensatzes zu verbessern. Dies wird erreicht, indem für jeden Commit, unabhängig von der Notwendigkeit, eine Mehrheitsentscheidung getroffen wird.

Für P3 könnte die Verwendung von Einzelumfragen eine geeignetere Alternative sein. Dabei erstellt der Projektinhaber eine authentifizierungsfreie Umfrage mit einer begrenzten Gültigkeitsdauer für einen Commit, der eine Mehrheitsentscheidung erfordert. Die Umfrage enthält alle relevanten Informationen zum Commit sowie Bewertungsoptionen für die einzelnen Dateien und ist über eine eindeutige URL zugänglich. Nach Abschluss der Umfrage können die Teilnehmer ihre Bewertungen unter ihrem Namen einreichen und ggf. später ändern, solange die Umfrage noch gültig ist. Der Projektinhaber kann den Fortschritt verfolgen, Schlüsse ziehen und den Commit entsprechend einstufen.

Obwohl die Einzelumfragen auf den ersten Blick praktisch erscheinen, da für die Teilnahme keine Registrierung in der Anwendung notwendig ist, weisen sie einige Mängel auf. Erstens ist die Sicherheit aufgrund der fehlenden Zugangskontrolle nicht gewährleistet, und zweitens können nicht alle Funktionen, die einem Projektmitglied zur Verfügung stehen, genutzt werden. Zum Beispiel können keine anderen Commits im Projekt eingesehen werden, die möglicherweise Ähnlichkeiten oder Bezüge zum aktuellen Commit haben. Aus diesen Gründen wird die Option nicht umgesetzt, aber einige wichtige Vorteile sollen trotzdem erreicht werden, indem jeder Commit im Projekt über eine eindeutige URL aufrufbar gemacht wird. Dadurch können auch Mitglieder, die sich nicht aktiv am Projekt beteiligen können, wertvolle Beiträge leisten, indem sie URLs zu den Commits erhalten, bei denen eine Mehrheitsentscheidung gefordert ist, und diese Commits dann überprüfen und bewerten.

4 Anforderungsanalyse

In diesem Kapitel werden aus den beschriebenen Lösungsansätzen konkrete Anforderungen an die Anwendung abgeleitet. Dazu wird eine *Anforderungsanalyse* durchgeführt, bei der die Anforderungen an eine Software erfasst, analysiert und modelliert werden. Sie ist häufig der erste Schritt in der Softwareentwicklung und dient dazu, sicherzustellen, dass die entstehende Software den Anforderungen der Benutzer entspricht. Eine Anforderungsanalyse kann dazu beitragen, Fehler beim Softwaredesign zu erkennen und zu vermeiden, indem sichergestellt wird, dass alle Anforderungen korrekt definiert und verstanden werden, bevor mit dem Design und der Implementierung der Software begonnen wird. Ohne eine gründliche Anforderungsanalyse besteht das Risiko, dass wichtige Anforderungen übersehen werden, was zu Fehlern und Mängeln in der resultierenden Software führen kann, oder dass während der Entwicklung Änderungen erforderlich werden, die wiederum Zusatzkosten verursachen können [18].

4.1 Funktionale Anforderungen

Dieser Abschnitt beschäftigt sich mit den funktionalen Anforderungen der Anwendung. Diese beschreiben, welche Funktionen und Leistungen die Anwendung erbringen muss, um die Bedürfnisse der Benutzer zu erfüllen. Dabei geht es darum, klar zu definieren, was die Anwendung tun soll, und nicht, wie sie es tun soll [18].

Der Abschnitt ist in 3 Unterabschnitte gegliedert: Rollen, Anforderungen und Berechtigungsüberblick. Unter dem Abschnitt „Rollen“ werden die verschiedenen Benutzergruppen aufgelistet, die auf die Anwendung zugreifen werden. Im Bereich „Anforderungen“ wird festgelegt, welche Funktionen die Anwendung für jede Rolle bereitstellen muss. In „Berechtigungsüberblick“ wird ein Überblick über die Zugriffsrechte jeder Rolle in der Anwendung gegeben.

4.1.1 Rollen

In den Lösungsansätzen lassen sich insgesamt fünf Benutzerrollen identifizieren. Die ersten drei sind typisch für Anwendungen mit Benutzerkontenverwaltung, während die restlichen beiden durch das zu implementierende RBAC definiert werden:

- Ein **unregistrierter Benutzer** hat Zugriff nur auf öffentliche Inhalte und Funktionen, für die kein Account benötigt wird (z.B. Anmeldeseite).

- Ein **registrierter Benutzer** kann seine Daten in seinem persönlichen Arbeitsbereich sichern und von dort aus verwalten sowie erweiterte Funktionen der Anwendung nutzen, indem er sich anmeldet.

- Ein **angemeldeter Benutzer** ist jemand, der sich erfolgreich bei der Anwendung registriert und sich durch Angabe seiner Anmeldedaten identifiziert hat. Dadurch hat er eine aktive Sitzung auf der Plattform und Zugang zu seinem persönlichen Arbeitsbereich.

- Ein **Projektmitglied** ist ein Benutzer, der als Mitglied an einem externen Projekt mitwirkt. Er besitzt nur eingeschränkte Zugriffsrechte, sodass er am Projekt mitarbeiten kann, aber keinen Zugriff auf die vertraulichen Bereiche und Funktionen hat.

- Ein **Projektinhaber** ist ein Benutzer, der ein Projekt erstellt hat und daran als Inhaber beteiligt ist. Er verfügt über alle möglichen Zugriffsrechte auf das Projekt, einschließlich der Rechte eines Projektmitglieds.

4.1.2 Anforderungen

Die folgende Auflistung zeigt alle relevanten Funktionen, die für die jeweilige Benutzerrolle erforderlich sind, um die gewünschten Aufgaben innerhalb der Anwendung erledigen zu können:

- *Registrierung & Anmeldung*: Ein unregistrierter Benutzer soll sich in der Anwendung registrieren können, um einen persönlichen Arbeitsplatz zu erstellen, auf dem er seine Projekte verwalten und alle verfügbaren Funktionen nutzen kann. Um einen schnellen Zugang zum Arbeitsplatz zu erhalten, muss eine einfache Authentifizierungsmethode vorhanden sein.

- *Projektverwaltung*: Ein angemeldeter Benutzer soll Projekte anlegen können, um Repositorys auf Sicherheitsfixes zu durchsuchen. Darüber hinaus soll er seine eigenen Projekte umbenennen, löschen und Projekte, an denen er nicht als Inhaber beteiligt ist, verlassen können. Eine übersichtliche Darstellung aller Projekte, an denen der Benutzer beteiligt ist, soll eine effiziente Projektverwaltung ermöglichen.

- *Commit-Filter*: Die Mitglieder eines Projekts sollen die Möglichkeit haben, Commits des Projekts aufzulisten und nach bestimmten Schlüsselwörtern in ihren Nachrichten zu filtern, um das Auffinden von Sicherheitsfixes zu erleichtern. Darüber hinaus soll ein Projektinhaber einen globalen Filter nach Dateierweiterungen anwenden können, um sicherzustellen, dass in seinem Projekt keine Dateien mit unerwünschten Erweiterungen erscheinen.

- *Commit-View*: Die Mitglieder eines Projekts sollen in der Lage sein, die in L1 aufgeführten Informationen zu einem Commit in einem übersichtlichen und strukturierten View, gemäß der Beschreibung in L2, einzusehen, um eine effektive Analyse durchführen zu können.

- *Commit-Slider*: Die Mitglieder eines Projekts sollen mithilfe eines in L2 definierten Sliders zwischen den verschiedenen Commit-Ansichten navigieren können, um den Zeitaufwand für die Navigation zu minimieren und eine effiziente Überprüfung der Commit-Änderungen zu erreichen.

- *Bewertungssystem*: Die Mitglieder eines Projekts sollen in der Lage sein, einzelne Commit-Dateien gemäß dem Bewertungsverfahren in L3 zu bewerten, um gemeinsam ihren Sicherheitsstatus zu bestimmen und die Dateien entsprechend zu verwenden. Zudem sollen die Mitglieder ihre Bewertungen einsehen und aktualisieren können.

- *Rollenbasiertes Zugriffssystem*: Es soll ein Zugriffssystem vorhanden sein, das eine zuverlässige Zusammenarbeit von mehreren Benutzern an einem Projekt ermöglicht und den Zugang zu bestimmten Projektfunktionen und -daten rollenbasiert regelt.

- *Mitgliederverwaltung*: Ein Projektinhaber soll eine Funktion haben, um eine übersichtliche Liste aller Mitglieder des Projekts anzuzeigen und somit einen Überblick über die Teilnehmer zu erhalten. Zudem soll es möglich sein, ein Mitglied aus dem Projekt zu entfernen, um ihm den Zugriff zu entziehen und seine Aktivitäten zu löschen.

- *Einladungssystem*: Ein Projektinhaber soll in der Lage sein, angemeldete Benutzer als Mitglieder zur gemeinsamen Arbeit an seinem Projekt einzuladen. Das System soll automatisch Benachrichtigungen an die eingeladenen Benutzer verschicken und ihnen die Möglichkeit geben, die Einladung einzusehen und darauf zu reagieren, indem sie sie annehmen oder ablehnen.
- *Exportfunktion*: Ein Projektinhaber soll die Möglichkeit haben, eine Exportdatei, deren Struktur und Inhalt in L3 festgelegt sind, generieren zu lassen, um sie zur Erstellung der benötigten Datensätze zu verwenden.
- *Accountverwaltung*: Ein angemeldeter Benutzer soll sein Passwort ändern können, um die Sicherheit des Accounts zu erhöhen oder die Anmeldung zu vereinfachen. Zudem soll er die Option haben, sein Account zu löschen, um seine Daten endgültig aus dem System zu entfernen.

4.1.3 Berechtigungsübersicht

Um einen Überblick über die Zugriffsrechte der Benutzerrollen zu haben, werden diese in einer Tabelle abgebildet:

Benutzerrolle	**Anforderungen**
Unregistrierter Benutzer	- Registrieren
Registrierter Benutzer	- Anmelden
Angemeldeter Benutzer	- Projekt erstellen - Projekte auflisten - Einladungen auflisten - Einladung annehmen - Einladung ablehnen - Passwort ändern - Account löschen
Projektmiglied	- Commits des Projekts auflisten - Commits nach Schlüsselwörtern filtern - Informationen zu einem Commit ansehen - Commit-Datei bewerten - Bewertung ändern - Projekt verlassen (falls nicht Inhaber)
Projektinhaber	- Mitglieder auflisten - Mitglied entfernen - Einladung verschicken - Commits global nach Dateierweiterungen filtern - Informationen zu den bewerteten Commits exportieren - Projekt löschen

Tabelle 1: Berechtigungsübersicht der Benutzerrollen

4.2 Nicht-funktionale Anforderungen

Nach der Festlegung der funktionalen Anforderungen, die die Kernfunktionen beschreiben, ist es an der Zeit, auf die nicht-funktionalen Anforderungen einzugehen. Anhand dieser Anforderungen werden die Qualitätsmerkmale definiert, die für einen erfolgreichen Einsatz der Anwendung und die Zufriedenheit der Benutzer unerlässlich sind [18]. In diesem Fall gibt es fünf solcher Anforderungen.

- *Leistung/Reaktionsfähigkeit*: Die Verarbeitung und Ausgabe von Daten soll möglichst schnell erfolgen, um den Benutzern Wartezeiten zu ersparen.

- *Zuverlässigkeit*: Alle Datenänderungen müssen im persistenten Speicher korrekt reflektiert werden. Parallele Schreibzugriffe dürfen nicht zu Datenverlusten oder Abstürzen führen. Zudem muss die Anwendung trotz fehlerhafter und unerwarteter Benutzereingaben stabil weiterlaufen können.

- *Verfügbarkeit:* Die Anwendung muss nahezu ständig verfügbar sein, um Benutzeranfragen zu verarbeiten und die gewünschten Ergebnisse zu liefern.

- *Usability*: Die Anwendung soll für die Benutzer einfach zu verstehen und zu benutzen sein und sie möglichst schnell an ihr Ziel bringen können.

- *Sicherheit*: Daten und Operationen, die nur für eine bestimmte Rolle oder einen bestimmten Benutzer vorgesehen sind, dürfen unter keinen Umständen für andere zugänglich sein.

- *Skalierbarkeit*: Die Anwendung soll in der Lage sein, verschiedene Arten von Lasten zu bewältigen, z.B. Datenvolumen, Anzahl der Benutzer oder Anzahl der Anfragen, ohne dass es zu einem Absturz oder einer Beeinträchtigung der Benutzererfahrung kommt.

- *Kompatibilität*: Die Anwendung muss auf gängigen Webbrowsern wie Google Chrome, Mozilla Firefox, Microsoft Edge und Safari ordnungsgemäß funktionieren.

4.3 Use-Case-Diagramm

Mithilfe eines Use-Case-Diagramms werden die Funktionen und die Benutzerrollen in eine klare Form überführt. Dabei handelt es sich um ein visuelles Modell zur Beschreibung der Funktionen und Interaktionen eines Systems. Der Einsatz eines Use-Case-Diagramms gibt einen Überblick über die Funktionen der Anwendung, macht Abhängigkeiten und Interaktionen zwischen ihnen deutlich und hilft sicherzustellen, dass alle erforderlichen Funktionen enthalten sind [18]. Die folgende Abbildung zeigt ein Use-Case-Diagramm der Anwendung:

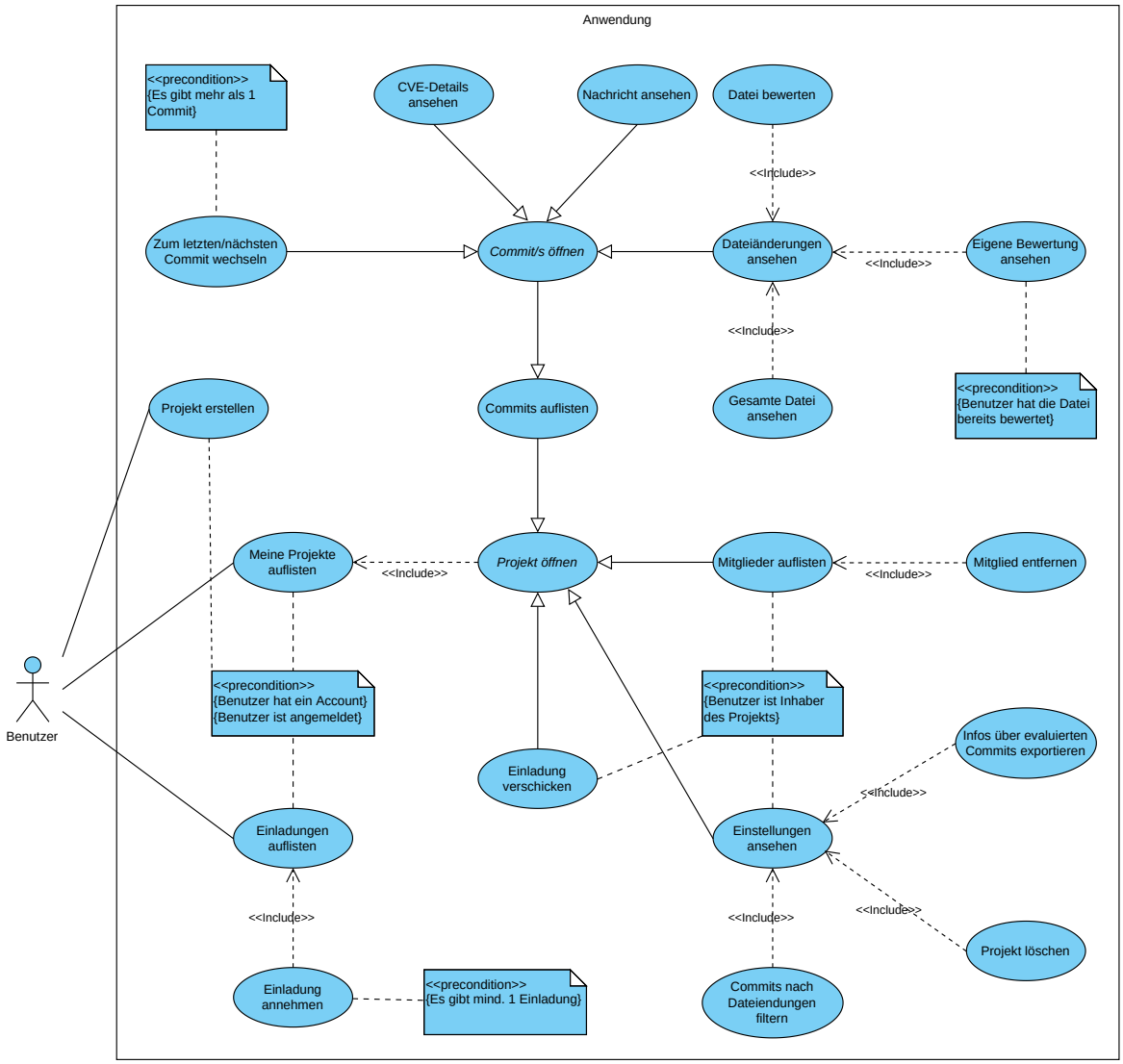

Abbildung 3: Use-Case-Diagramm der Anwendung

5 Konzeption und Implementierung

In diesem Kapitel werden die Ergebnisse der Anforderungsanalyse genutzt, um eine detaillierte Konzeption der Anwendung zu entwickeln. Die Konzeptionsphase stellt einen kritischen Schritt in der Softwareentwicklung dar. Während die Anforderungsanalyse den Zweck der Software und die Erwartungen der Benutzer definiert, konzentriert sich die Konzeption auf die Realisierung dieser Anforderungen. Eine sorgfältige und systematische Konzeption legt den Grundstein für eine effektive Umsetzung und bietet klare Leitlinien für die gesamte Entwicklungsphase.

5.1 Zugriff auf Informationen

Da die Bereitstellung von Informationen die Hauptfunktion der Anwendung ist, ist es wichtig, dass sie effizient ausgeführt wird. In diesem Abschnitt werden Entscheidungen bzgl. der Art und Weise des Zugriffs auf die einzelnen Informationen getroffen. Um zu einer objektiven und nachvollziehbaren Entscheidung zu gelangen, werden verschiedene Methoden vorgestellt und anhand eines Vergleichs die bestmögliche Lösung ausgewählt. Auf der Basis der gewählten Lösungen wird später die Datenbank entworfen (siehe Abschnitt 5.2.3).

Commits

Um den Zugriff auf Commit-Nachrichten effizient zu gestalten, reicht es aus, sie aus Commits zu extrahieren und in einer Datenbank abzulegen. Die Größe von Commit-Nachrichten ist in der Regel relativ gering, sodass sie schnell aus der Datenbank abgefragt werden können.

Bei Commit-Änderungen stellt sich die Situation anders dar. Für den Zugriff werden zunächst die geänderten Zeilen der jeweiligen Datei benötigt. Um diese Voraussetzung zu erfüllen, können *Patches* verwendet werden. Ein Patch ist eine kompakte Darstellung der Unterschiede zwischen zwei Dateien, die sowohl die alte als auch die neue Version der modifizierten Zeilen einschließt. Darüber hinaus enthält er Kontextzeilen und Zeilennummern, die Auskunft über die Position in der Datei liefern. Im Vergleich zum Patch stellt ein *Diff* hauptsächlich nur die Unterschiede dar. Mit *Git*, einem Tool zur verteilten Versionsverwaltung von Dateien, kann ein Patch zwischen zwei beliebigen

Commits erstellt werden [19]. Die Frage ist dabei, wie auf diese Patches zugegriffen werden kann, ohne die Leistung und Skalierbarkeit der Anwendung zu beeinträchtigen.

Eine Variante besteht darin, Patches unmittelbar beim Abruf zu erzeugen. Diese Lösung ist nicht besonders speicherintensiv, hat aber Schwächen in Bezug auf die Geschwindigkeit. Das liegt an der Art und Weise, wie Git mit Dateiänderungen umgeht: Anstatt nur die Änderungen zu erfassen, wird ein Snapshot der aktuellen Datei erstellt, mit einem verlustfreien Algorithmus komprimiert und im Dateisystem abgelegt. Wenn ein Patch benötigt wird, nimmt Git die beiden Versionen der Dateien, dekomprimiert sie, berechnet die Unterschiede, formatiert sie gemäß dem Patch-Format und gibt das Resultat zurück [19]. Angesichts dieses kostenaufwendigen Prozesses und der Tatsache, dass die Patches in der Anwendung über das Netzwerk übertragen werden müssen, kann der Benutzer beim Navigieren durch die Commits große Verzögerungen erfahren, was gegen die Usability-Anforderung verstößt.

Bessere Zugriffszeiten können erreicht werden, indem alle Patches einmal erzeugt und als Dateien im System, für den späteren Zugriff, aufbewahrt werden. Um eine Vorstellung vom Ausmaß der Verbesserung zu bekommen, wurde die Zeit mit einem Python-Skript gemessen, das in einer Schleife, je nach Zugriffsvariante, Patches einzeln liest oder erzeugt. Die Wahl von Python ist dabei eine bewusste Entscheidung, da diese Programmiersprache auch im Backend der Anwendung verwendet wird (siehe Abschnitt 5.2.1). Ein Vergleich zeigt, dass die Zeiten beim Zugriff über das Dateisystem wesentlich kürzer sind:

Anzahl der Commits	100	200	300
per Erzeugung	3.635 s	7.417 s	11.465 s
per Dateisystem	0.006 s	0.012 s	0.020 s

Tabelle 2: Zeiten für den Zugriff auf Commits

Eine erweiterte Form des letzteren ist die Aufbewahrung von Patches in einer relationalen Datenbank. Für die Anwendung hat es zwei wichtige Vorteile: Erstens wird die flexible SQL-Sprache zur Verfügung gestellt, die das Abrufen und Verwalten von Patches erleichtert, und zweitens verringert sich die Anzahl der notwendigen Schritte für den Zugriff auf einen Patch. Für den direkten Zugriff über ein Dateisystem müsste die Anwendung die Pfade von Patchdateien in einer Datenbank führen, um diese lokalisieren zu können. Das bedeutet, dass für den Zugriff auf eine Patchdatei

sowohl ein Datenbank- als auch ein Dateisystemzugriff erforderlich wären. Im Falle dynamisch gebildeter Dateipfade wären es sogar mehrere Datenbankzugriffe. Daher ist die Ablage in einer relationalen Datenbank sehr hilfreich, denn dadurch werden all diese Zugriffe auf einen einzigen reduziert. Zusätzlich erleichtert es die Erstellung von Sicherungskopien, da alle Daten in einer Datenbank enthalten sind.

Die Patches können in ihrer Form nur schlecht verarbeitet werden, was die Implementierung und den Betrieb des Dateierweiterungsfilters erschwert. Um auf das Diff bzw. die Änderungen einer relevanten Datei zuzugreifen, müsste der entsprechende Patch erst gefunden, vollständig eingelesen und geparst werden, was zu einem erhöhten Speicherverbrauch und Leistungseinbußen führen könnte. Das Problem wäre vor allem dann groß, wenn der Patch eine oder mehrere komplette, irrelevante Dateien einschließt. Die erste Maßnahme ist daher die Aufteilung von Patches in einzelne Diffs. Eine hohe Anzahl von resultierenden Diffs kann Auswirkungen auf die Zugriffszeit haben, aber dies kann durch den Einsatz von Datenbankindizes minimiert werden [20].

Die Größe eines einzelnen Diffs entspricht einem optimalen Wert, aber ihre Gesamtgröße kann für eine Datenbank beträchtlich sein. Um diese zu reduzieren, werden Diffs jeweils mit *zlib* komprimiert, einer quelloffenen Bibliothek zur verlustfreien Kompression und Dekompression von Daten. Was die Zeitkosten anbelangt, verfügt zlib über 10 Kompressionsstufen mit jeweils unterschiedlicher Geschwindigkeit und Kompressionsrate. Stufe 1 steht für die schnellste Kompression mit der niedrigsten Rate, Stufe 9 für die langsamste Kompression mit der höchsten Rate und Stufe 6 für einen Ausgleich zwischen Geschwindigkeit und der Rate [21]. Bei der Kompression von Diffs wird die Stufe 6 verwendet, die dann je nach Bedarf erhöht oder gesenkt werden kann.

n=3000	⌀Patch-Größe	Anzahl der Diffs	⌀Diff-Größe	⌀Diff-Größe (zlib)	Gesamtgröße	Gesamtgröße (zlib)
postgres	17,92 KB	18997	2,83 KB	0,79 KB	53,77 MB	14,99 MB
curl	7,27 KB	16061	1,36 KB	0,51 KB	21,81 MB	8,25 MB
wildfly	29,55 KB	41252	2,15 KB	0,58 KB	88,65 MB	24,1 MB
kubernetes	60,32 KB	30776	5,88 KB	1,19 KB	180,95 MB	36,5 MB
httpd	31,99 KB	19794	4,85 KB	1,17 KB	95,96 MB	23,1 MB
openssl	7,08 KB	10308	2,06 KB	0,66 KB	21,23 MB	6,78 MB
systemd	10,35 KB	12981	2,39 KB	0,69 KB	31,04 MB	8,92 MB
django	12,87 KB	13890	2,78 KB	0,74 KB	38,61 MB	10,25 MB

Tabelle 3: Statistische Daten (Kompressionsstufe=6)

CVEs

Für den Zugriff auf CVE-Informationen besteht die erste Möglichkeit darin, eine Sammlung von CVE-Einträgen für einen bestimmten Zeitraum herunterzuladen, sie in eine Datenbank zu verschieben und dann diese Datenbank zu verwenden. Dies hat den Vorteil, dass die Abhängigkeit von externen Quellen entfällt, was den Zugriff vereinfacht und beschleunigt. Der Nachteil ist jedoch, dass die Datenbank viel Speicherplatz beansprucht und ständig mit neuen CVE-Einträgen aktualisiert werden muss, um sie auf dem neusten Stand zu halten.

Die zweite Option ist, die Informationen bei Bedarf über eine CVE-API abzufragen. Da die API eine Verbindung zu einer zentralen Datenquelle herstellt, lassen sich die aktuellsten Informationen zu jedem CVE-Eintrag finden. Dieser Ansatz ist vor allem kosteneffizient, da keine große Menge an Daten heruntergeladen und aufbewahrt werden muss. Andererseits gibt es aber auch Nachteile: Wenn die API ausfällt oder nicht mehr unterstützt wird, kann dies Verfügbarkeitsprobleme nach sich ziehen. Die API kann aber auch Einschränkungen haben, die auf verschiedene Weise auftreten können, wie z.B. Limitierungen bei der Anzahl der Anfragen, die pro Zeiteinheit gestellt werden können, oder bei der Größe oder Art der Daten, auf die zugegriffen werden kann. Weiterhin kann die *SOP* (*same-origin policy*) bei modernen Webbrowsern Zugriffsprobleme bereiten. Wenn eine Webseite auf Ressourcen zugreifen möchte, die von einer anderen Quelle stammen (d.h. eine andere Domäne, Protokoll oder Port haben), tritt die SOP-Sicherheitspolitik in Kraft. Dies bedeutet, dass der Zugriff standardmäßig blockiert wird, um zu verhindern, dass bösartige Skripte sensible Daten weiterreichen oder andere Malware laden [22]. Mit dem *CORS* (*Cross-Origin Resource Sharing*), das die Kommunikation mit externen Quellen ermöglicht, kann dieses Hindernis umgangen werden [23]. Allerdings muss es von beiden Parteien unterstützt werden, was wiederum nicht immer der Fall ist.

Die letzte Option ist, die API zu verwenden, um Informationen ausschließlich über die vorhandenen CVE-Identifikatoren zu erhalten und sie in einer Datenbank abzulegen. Diese Option vereint die Vorteile der ersten beiden Optionen. Zum einen wird Zeit gespart, indem ständige API-Abfragen vermieden werden, und zum anderen wird sichergestellt, dass die Informationen immer verfügbar sind, sobald sie einmal extrahiert wurden. Zu den Mängeln gehört die Tatsache, dass die API zum Zeitpunkt der

Extraktion immer noch versagen kann, und dass die extrahierten Informationen in bestimmten Intervallen aktualisiert werden müssen, da sie sich bekanntlich mit der Zeit ergänzen können (siehe Abschnitt 2.4).

Nach dem Vergleich aller Optionen zeigt sich, dass die letztere Option für die Bereitstellung von CVE-Informationen besser geeignet ist, da sie ein ausgewogenes Verhältnis zwischen Zugriffsgeschwindigkeit und Speicheraufwand bietet.

5.2 Architektur

In diesem Kapitel wird die Anwendungsarchitektur ausgearbeitet. Architekturplanung ist ein Prozess, bei dem die wesentlichen Eigenschaften einer zukünftigen Software, wie z.B. ihre Struktur, Komponenten, Interaktionen und Verhaltensmuster, geplant und definiert werden. Dieser Prozess ist von entscheidender Bedeutung, da er die Basis für die Umsetzung der Anforderungen bildet. Eine gute Architektur erleichtert künftige Änderungen und Erweiterungen und verbessert gleichzeitig die Effizienz und Zuverlässigkeit der Software. Eine schlechte Architektur hingegen kann zu einem komplexen und kaum verständlichen System führen, das schwer zu pflegen und zu warten ist. Außerdem kann es zu Leistungsproblemen und schlechter Bedienbarkeit kommen [24].

5.2.1 Technologien

Die Wahl der richtigen Technologien für die Softwareentwicklung ist äußerst wichtig, da sie einen großen Einfluss auf die Qualitätsmerkmale der Software haben kann, z.B. Leistung, Wartbarkeit und Sicherheit. Eine geeignete Technologieauswahl kann auch die Entwicklungseffizienz erheblich verbessern. In diesem Abschnitt werden die ausgewählten Technologien, die Gründe für ihre Auswahl sowie ihr Nutzen für die Anwendung vorgestellt.

Für das Backend wird die Programmiersprache *Python* verwendet. Sie hat eine leicht lesbare und verständliche Syntax, was die Entwicklung und das Debugging vereinfacht. Da es sich um eine interpretierte Sprache handelt, kann der Code während der Entwicklung schnell und einfach getestet werden [25]. Python verfügt zudem über eine umfangreiche Standardbibliothek und eine Vielzahl von externen Bibliotheken, die von der Python-Community bereitgestellt werden.

Für den Datenaustausch zwischen dem Client und dem Server wird eine REST-API umgesetzt. Sie ermöglicht den Zugriff auf die Ressourcen einer Anwendung über eine einfache und standardisierte Schnittstelle, was die Anwendungsentwicklung vereinfacht und die Interoperabilität verbessert. Eine solche API beruht auf klaren und einfachen Prinzipien, die eine hohe Flexibilität und Skalierbarkeit bieten (siehe Abschnitt 2.1), und ist aufgrund der Verwendung von standardmäßigen Protokollen und Formaten einfach zu implementieren und zu testen. Um die API vor unberechtigtem Zugriff zu schützen, wird sie gemäß dem Standard OAuth 2.0 abgesichert (siehe Abschnitt 2.2).

Die Daten werden im *JSON*-Format ausgetauscht. Es ist ein textbasiertes Datenformat, das zum Speichern und Übertragen von strukturierten Daten verwendet wird. Die Daten in JSON bestehen aus einer Sammlung von Schlüssel-Wert-Paaren, die zusammen ein JSON-Objekt bilden. Bei den Werten kann es sich um Zeichenketten, Zahlen, boolesche Werte, Arrays oder andere JSON-Objekte handeln [16]. Das Format eignet sich besonders gut für REST-APIs, da leichtgewichtig und einfach zu verarbeiten ist. Im Vergleich zu anderen Datenformaten wie XML ist JSON weniger datenintensiv und leichter zu parsen, sodass es von Client- und Serveranwendungen effizient verarbeitet werden kann [26].

Die REST-API wird mit *Flask* umgesetzt. Dies ist ein Python-Web-Framework, das darauf ausgelegt ist, Webanwendungen und APIs schnell und einfach zu entwickeln. Es ist ein minimalistisches Framework, das auf den Grundfunktionen aufbaut, aber gleichzeitig flexibel genug ist, um die Anwendung entsprechend den Anforderungen zu gestalten. Die Grundfunktionen ermöglichen das Routing und die Behandlung von HTTP-Anfragen sowie die Integration von Templates und Datenbanken. Durch die Verwendung von Extensions können zusätzliche Funktionen und Technologien integriert werden, darunter auch die OAuth [27].

Die Daten werden in einer *SQLite*-Datenbank gespeichert. SQLite ist eine relationale Datenbank-Engine, die eine einfache und effiziente Möglichkeit zum Speichern und Abrufen von Daten in einer dateibasierten Datenbank bietet. Im Gegensatz zu anderen Datenbank-Engines wie MySQL oder PostgreSQL muss für SQLite kein separates Datenbanksystem installiert werden, da SQLite als eigenständige Bibliothek direkt in die Anwendung eingebettet wird. Dadurch entfällt die Notwendigkeit, Daten über mehrere Prozesse oder Server hinweg auszutauschen, was die Leistung der Anwendung

deutlich verbessert. Andererseits ist die Mehrbenutzerfähigkeit aufgrund der fehlenden ausgefeilten Zugriffskontrolle eingeschränkt [28]. Es ist jedoch nicht zu erwarten, dass eine große Anzahl von Benutzern die Anwendung gleichzeitig nutzen wird, daher sollte dies kein Problem darstellen.

Für das Frontend wird hauptsächlich React zusammen mit *Material-UI* verwendet. React ist eine JavaScript-Bibliothek, die für die Erstellung von Benutzeroberflächen eingesetzt wird (siehe Abschnitt 2.3). Die Vorteile von React sind eine bessere Wiederverwendbarkeit des Codes, eine höhere Produktivität, eine einfachere Wartung und ein schnelleres Rendering von Benutzeroberflächen. Material-UI (MUI) ist eine React-Komponentenbibliothek, die vorgefertigte und flexible Komponenten und Designs für Benutzeroberflächen enthält. Sie kann die Entwicklung von Benutzeroberflächen signifikant beschleunigen, da nicht jede Komponente von Grund auf neu erstellt werden muss. Material UI umfasst eine Vielzahl von grafischen Elementen, darunter Buttons, Textfelder, Dropdown-Menüs, Icons, Dialoge und viele andere [29].

5.2.2 Aufbau

Nach einer Einführung in die verwendeten Technologien und Werkzeuge soll gezeigt werden, wie sie zur Entwicklung einer stabilen und skalierbaren Anwendung genutzt werden können. Die folgende Abbildung gibt einen groben Überblick über die Hauptkomponenten der Anwendungsarchitektur sowie deren Zusammenspiel:

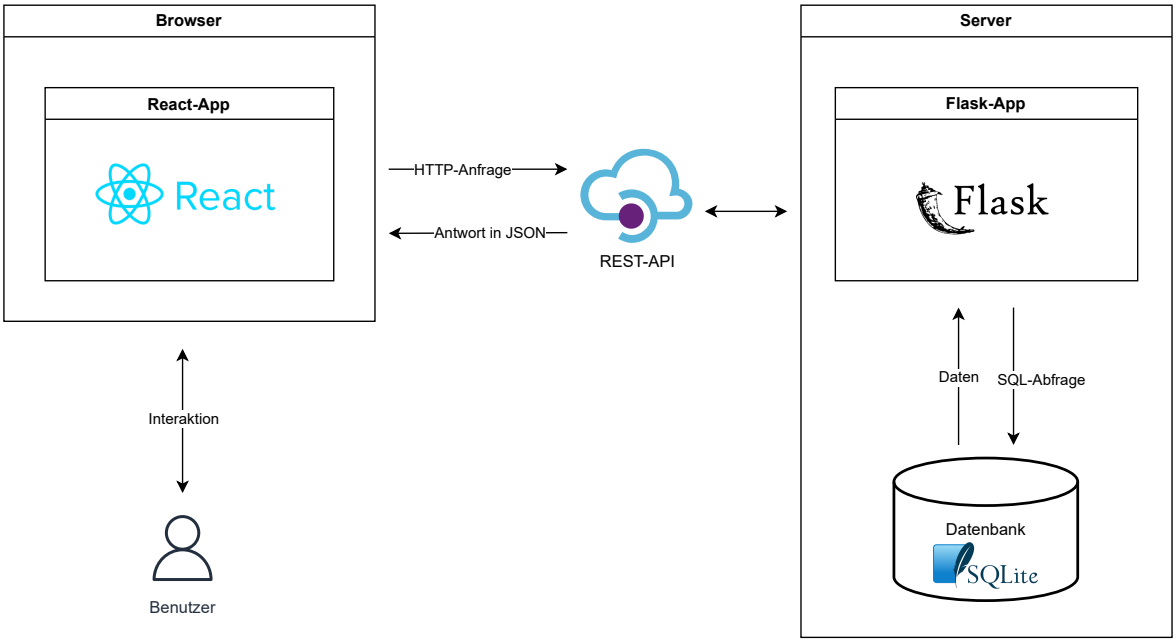

Abbildung 4: Aufbau der Anwendungsarchitektur

Die Architektur verwendet das Client-Server-Modell, um Ressourcen zu zentralisieren und sie für mehrere Benutzer zugänglich zu machen. Es gibt ein Programm, das auf einem entfernten Rechner läuft und Dienste über das HTTP-Protokoll bereitstellt, und eines, das auf einem Browser des Benutzers läuft und diese Dienste konsumiert.

Der Benutzer bedient die Anwendung über einen Webbrowser. Dabei interagiert er unmittelbar mit der *React-App*. Die Komponente verarbeitet die vom Benutzer ausgelösten Ereignisse, indem sie für jedes Ereignis eine bestimmte Geschäftsaktion (z.B. Projekterstellung) ausführt und die UI dann entsprechend anpasst. Sie umfasst somit die Logik und die Benutzeroberfläche. Um eine Geschäftsaktion abzuwickeln, werden meist zusätzliche Daten vom Server oder die Persistenz von neuen Daten auf dem Server benötigt. Zu diesem Zweck gibt es die *Flask-App*-Komponente, die eine REST-API zum Datenabruf und -verwaltung bereitstellt. Bei Bedarf stellt die *React-App* eine Anfrage mit Angabe der URI einer Ressource und der gewünschten CRUD-Operation. Die *Flask-App* empfängt sie, verarbeitet sie und kann dabei Daten aus der Datenbank beziehen. Im Anschluss daran liefert sie eine Antwort mit einem Statuscode und Daten im JSON-Format (falls vorhanden).

5.2.3 Datenmodell

Ein Datenmodell ist eine abstrakte Darstellung der in einer Anwendung geführten Daten. Es beschreibt die Struktur und die Beziehungen zwischen den Daten und dient als Grundlage für die Implementierung der Datenbank [24]. Der nächste Schritt ist somit der Entwurf eines Datenmodells, das eine konsistente und korrekte Speicherung der Daten in der Anwendung gewährleistet und eine schnelle Abfrage und Verarbeitung ermöglicht.

Aus den Anforderungen lässt sich schließen, dass die zu verwaltenden Datenstrukturen und ihre Beziehungen nicht besonders komplex sind. Aus diesem Grund kann auf die Erstellung eines ER-Diagramms (Entity-Relationship-Diagramms) verzichtet werden und gleich ein detailliertes Datenbankschema definiert werden. Das folgende Datenbankschema zeigt die Struktur und Organisation der Daten in der Datenbank:

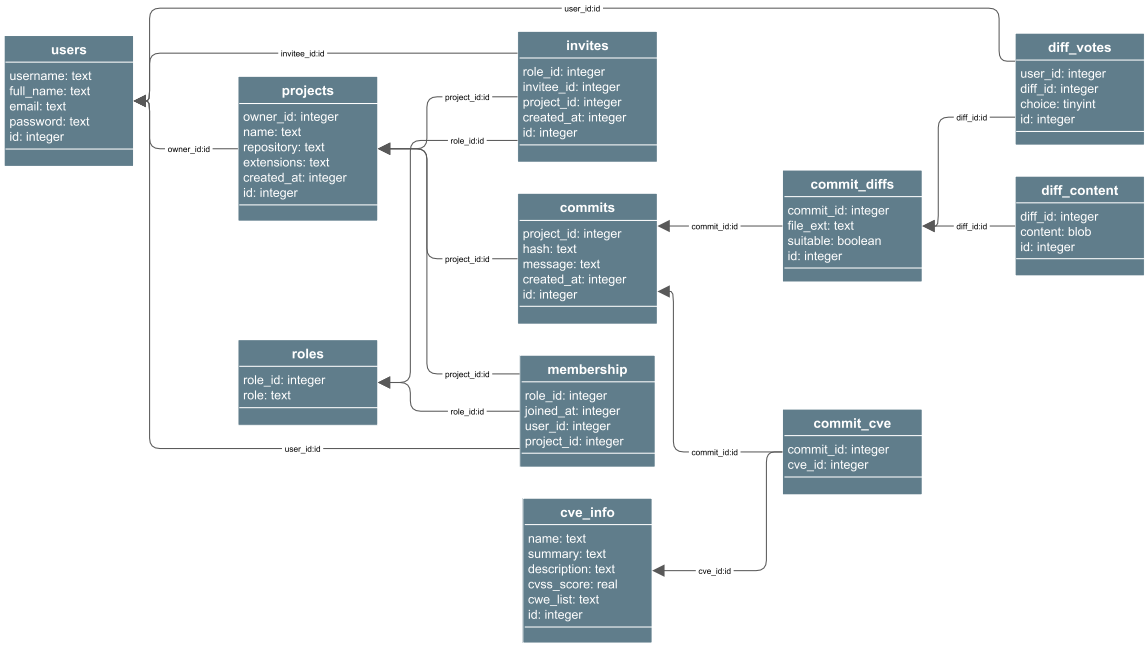

Abbildung 5: Datenbankschema

Die Datenbank befindet sich in der 3. Normalform (3NF), d.h. es gibt keine mehrwertigen Attribute und jedes Nicht-Schlüssel-Attribut hängt funktional bzw. logisch nur vom Primärschlüssel ab. Der Hauptnutzen der 3NF ist die Vermeidung von Redundanzen in der Datenbank. Durch die Eliminierung redundanter Daten verringert sich der Speicherbedarf sowie die Wahrscheinlichkeit von Inkonsistenzen in den Daten. In der 3NF werden transitive Abhängigkeiten in separate Tabellen ausgelagert und mit Fremdschlüsselbeziehungen verknüpft. Diese Trennung von verbundenen, aber unabhängigen Daten erhöht die Datenkonsistenz und vereinfacht die Datenbankwartung, da Änderungen nur an einer Stelle vorgenommen werden müssen [30].

Die Datensätze in der Datenbank werden mit Surrogatschlüsseln (*id*-Attribut) identifiziert. Ein Surrogatschlüssel ist ein künstlicher Schlüssel, der keine semantische Bedeutung trägt. Ein natürlicher Schlüssel hingegen wird aus einem oder mehreren Attributen der Datensätze abgeleitet [31]. In der Datenbank werden ganzzahlige Surrogatschlüssel verwendet, da sie mehrere wichtige Vorteile haben. Einer der wichtigsten Vorteile ist, dass ihre Eindeutigkeit, durch die Erzeugung von der Datenbank selbst, garantiert ist. Zudem werden sie im Laufe der Zeit nicht verändert und sind daher stabil. Letztlich sind sie nicht so speicherintensiv wie natürliche Schlüssel (z.B. der Benutzername) und können daher von der Datenbank effizienter verarbeitet werden.

Es ist zu bemerken, dass Diff-Details und Diff-Inhalte in separaten Tabellen gehalten werden. Der Grund dafür ist, dass Diff-Inhalte in der Regel recht groß sind, und wenn sie zusammen mit häufig abgerufenen Daten in einer Tabelle abgelegt werden, kann dies den Zugriff auf diese Daten beeinträchtigen. Daher hilft die sogenannte vertikale Partitionierung, in diesem Fall eine Unterteilung in Daten und Metadaten, dieses potenzielle Problem zu beseitigen [32].

Im Folgenden wird der Zweck der einzelnen Tabellen in Abbildung 5 dargelegt.

- **users** - Speichert Daten der registrierten Benutzer. Dazu gehören ein Benutzername (*username*), ein vollständiger Name (*full_name*) und ein gehashtes Passwort (*password*).

- **projects** - Speichert Informationen über die bestehenden Projekte. Ein Datensatz enthält einen Projektnamen (*name*), die URN des geparsten Repositorys (*repository*), eine kommagetrennte Liste von Dateierweiterungen (*extensions*) und die ID des Inhabers (*owner_id*).

- **invitations** - Speichert Informationen über die versendeten Einladungen. Ein Datensatz enthält die ID des eingeladenen Benutzers (*invitee_id*), die ID des Projekts, zu dem er eingeladen wurde (*project_id*), und die ihm zugewiesene Rolle im Projekt (*role_id*). Durch das Feld *role* soll die Erweiterung von Rollen in der Zukunft ermöglicht werden.

- **commits** - Speichert Details von Commits bzw. potenziellen Sicherheitsfixes, die in Repositorys entdeckt wurden.

- **cve_info** - Speichert Details von CVE-Einträgen, die in Commit-Nachrichten gefunden wurden.

- **membership** - Speichert Informationen darüber, wer an welchem Projekt beteiligt ist und welche Rolle er darin hat.

- **commit_diffs** - Speichert Informationen über die Commit-Diffs. Ein Datensatz besteht aus der ID des zugehörigen Commits (*commit_id*), der Erweiterung der Diff-Datei (*file_ext*) und einem booleschen Wert, der bestimmt, ob das Diff den Benutzern angezeigt wird (*suitable*).

- **diff_content** - Speichert Inhalte der Diffs. Der Inhalt eines Diffs wird als eine Menge von Bytes im Feld *content* aufbewahrt.

- **commit_cve** - Speichert Informationen darüber, welcher CVE-Eintrag zu welchem Commit gehört.

- **votes** - Speichert Bewertungen der Diffs. Das Feld *diff_id* enthält die ID des betreffenden Diffs, *user_id* die ID des Begutachters und *choice* eine der folgenden Zahlen, von denen jede eine bestimmte Bedeutung trägt: -1 (nicht verwundbar), 0 (neutral), 1 (verwundbar).

- **roles** - Speichert die vorhandenen Rollen als Einzelbuchstaben, z.B. „M“ für „Member“ und „O“ für „Owner“.

5.3 Grafische Oberfläche

Die Implementierung der grafischen Benutzeroberfläche der Anwendung soll in Anlehnung an die Norm DIN EN ISO 9241-110 erfolgen. Diese befasst sich mit der ergonomischen Gestaltung von interaktiven Systemen und definiert 7 Prinzipien, die eine benutzerfreundliche UI ausmachen [33]. Diese Prinzipien werden bei der Entwicklung berücksichtigt und später als Grundlage für die Evaluation herangezogen:

1. *Aufgabenangemessenheit*: Das System sollte auf die Aufgaben und Ziele der Benutzer ausgerichtet sein und ihnen helfen, ihre Aufgaben effektiv und effizient zu erledigen. Es sollte die notwendigen Funktionen bereitstellen und gleichzeitig überflüssige Aktionen oder Informationen vermeiden.

2. *Selbstbeschreibungsfähigkeit*: Das System sollte den Benutzern klar und verständlich anzeigen, was es tut und wie es funktioniert. Sie sollten immer über den aktuellen Systemstatus informiert sein und wissen, welche Aktionen sie ausführen können.

3. *Erwartungskonformität*: Das System sollte den Erwartungen der Benutzer entsprechen und sich an gemeinsame Standards und Konventionen halten. Die Benutzer sollten in der Lage sein, das System anhand ihrer bisherigen Erfahrungen mit ähnlichen Systemen zu bedienen.

4. *Lernförderlichkeit*: Das System sollte für die Benutzer einfach zu erlernen sein. Es sollte leicht verständliche Anleitungen, Hilfestellungen und Rückmeldungen bieten, um den Benutzern zu helfen, sich mit dem System vertraut zu machen und ihre Kenntnisse und Fähigkeiten zu erweitern.

5. *Steuerbarkeit*: Die Benutzer sollten die volle Kontrolle über das System haben und es nach ihren Bedürfnissen steuern können. Das System sollte flexible Interaktionsmöglichkeiten bieten und den Benutzern erlauben, Aktionen rückgängig zu machen oder abzubrechen.

6. *Fehlertoleranz*: Das System sollte so gestaltet sein, dass die Benutzer Fehler ohne schwerwiegende Konsequenzen machen können. Es sollte Fehler erkennen, verhindern oder korrigieren und den Benutzern helfen, ihre Ziele zu erreichen, auch wenn Fehler auftreten.

7. *Individualisierbarkeit*: Das System sollte es den Benutzern ermöglichen, die Benutzeroberfläche an ihre individuellen Bedürfnisse und Vorlieben anzupassen. Es sollte Optionen für die Anpassung von Einstellungen, Layouts oder Darstellungen bieten, um eine personalisierte Benutzererfahrung zu erzielen [33].

5.4 Entwicklungsansatz

Für die Entwicklung der konzipierten Anwendung wird *Prototyping* zugrunde gelegt. Dies ist ein Ansatz in der Softwarentwicklung, bei dem ein vorläufiges, teilweise funktionsfähiges Modell bzw. ein Prototyp einer Software erstellt wird, um Designkonzepte, Funktionen und Benutzerfreundlichkeit zu testen und zu validieren. Es handelt sich dabei um eine iterative Methode, die Feedback und Anpassungen in einem frühen Stadium der Entwicklung ermöglicht. Auf diese Weise können mögliche Verbesserungen noch vor der endgültigen Implementierung erkannt und umgesetzt werden. Darüber hinaus können potenzielle Risiken und Probleme identifiziert und beseitigt werden, bevor größere Ressourcen aufgewendet werden (z.B. Entwicklungszeit) [34].

Es gibt verschiedene Arten von Prototyping, darunter vertikales und horizontales. Beim vertikalen Prototyping liegt der Schwerpunkt auf der Entwicklung eines voll funktionsfähigen Teils des Systems. Dabei werden die Hauptfunktionen priorisiert und detailliert ausgearbeitet. Ziel ist es, ein Verständnis für die Benutzerinteraktion und die wichtigsten technischen Aspekte zu gewinnen. Horizontales Prototyping hingegen bezieht sich auf die Entwicklung und den Test einer einzelnen kompletten Schicht, die einen bestimmten Funktionsbereich abdeckt. Die verbleibenden Schichten werden dabei als Stubs oder Mocks simuliert. Ziel ist es, das Zusammenspiel verschiedener Funktionen oder Module zu testen und sicherzustellen, dass sie miteinander kompatibel sind. Dabei geht es weniger um die konkreten Details der einzelnen Funktionen als vielmehr um die Interaktion und Integration zwischen ihnen [34].

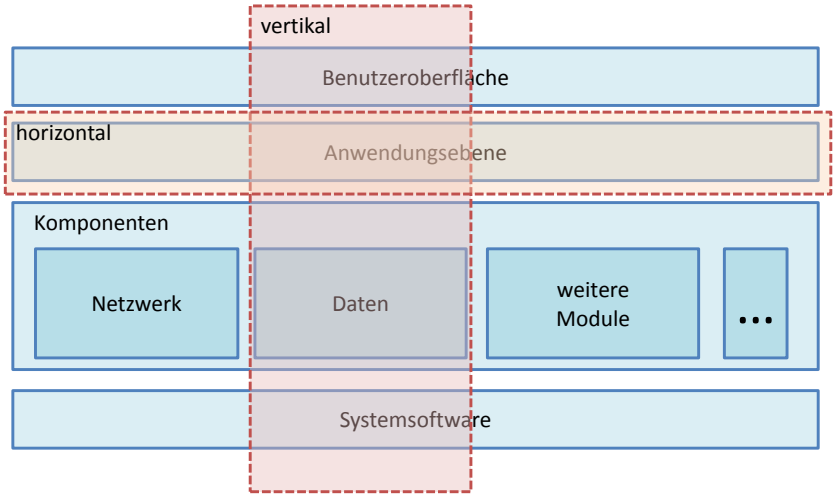

Abbildung 6: Visualisierung von vertikalem und horizontalem Prototyping [35]

Bei der Entwicklung der Anwendung wird zunächst das horizontale Prototyping eingesetzt, um das Design der Benutzeroberfläche grob zu entwerfen und zu überprüfen. Anschließend wird nach dem vertikalen Prototyping vorgegangen, indem die Funktionen der Anwendung nacheinander implementiert, getestet und verbessert werden. Der Grund für die Wahl des vertikalen Prototypings ist, dass so technische Herausforderungen frühzeitig erkannt und gelöst werden können. Durch die Implementierung von Kernfunktionen können die Realisierbarkeit überprüft und Risiken in Bezug auf die Technologie oder die Leistung der Anwendung identifiziert werden. Dadurch lassen sich dann Designentscheidungen anpassen oder alternative Ansätze finden, was beim horizontalen Prototyping nur schwer möglich ist [34].

6 Evaluation

Nach der Implementierung der Anwendung wird diese evaluiert, um die im Abschnitt 1.3 festgelegten Forschungsfragen zu beantworten und das System auf seine Eignung zu prüfen. Die Evaluation ist ein Prozess, der systematisch und gründlich durchgeführt wird, um festzustellen, ob die Anwendung die gesetzten Ziele und Anforderungen erfüllt. Durch eine sorgfältige Evaluation können Stärken und Schwächen des Systems ermittelt werden, um in Zukunft Verbesserungen vorzunehmen und eine optimale Nutzung der Anwendung zu gewährleisten.

6.1 Methode

Für die Evaluation und Beantwortung der Forschungsfragen müssen Daten gesammelt, analysiert und interpretiert werden. Für diesen Zweck wurden zwei grundlegende Arten der empirischen Forschung in Betracht gezogen: *quantitative* und *qualitative*. Bei der quantitativen Forschung geht es um die Erfassung und Analyse von numerischen Daten, die mit statistischen Verfahren quantifiziert werden können. Im Rahmen solcher Studien werden oft strukturierte Fragebögen, Umfragen oder experimentelle Methoden verwendet, um Daten von einer großen Anzahl an Teilnehmern zu sammeln. Im Gegensatz dazu befasst sich die qualitative Forschung mit der Erhebung und Analyse nicht-numerischer Daten, um ein tieferes Verständnis von Phänomenen, Erfahrungen und Kontexten zu gewinnen. Qualitative Methoden umfassen oft Interviews, Beobachtungen, Fokusgruppen oder die Analyse von Dokumenten und Artefakten [36]. Für das verfolgte Ziel wurde die qualitative Forschungsart gewählt. Der Grund dafür ist, dass die Teilnehmerzahl für die Durchführung einer quantitativen Studie aufgrund des eingeschränkten Zugangs zur Zielgruppe und der begrenzten Ressourcen nicht ausreichte. Bei quantitativen Studien ist es wichtig, eine möglichst große Stichprobe zu haben, um statistisch aussagekräftige Ergebnisse zu erhalten und Verallgemeinerungen vornehmen zu können. Wenn die Teilnehmerzahl zu gering ist, kann dies die Gültigkeit und Zuverlässigkeit der Ergebnisse beeinträchtigen [37].

Zur Datenerhebung für die zweite Forschungsfrage bzgl. der notwendigen Eigenschaften einer Anwendung, um Sicherheitsfixes auf eine benutzerfreundliche Weise zu erhalten, wurde *Usability-Testing* angewandt. Dies ist eine Methode zur Evaluation eines Produkts oder einer Dienstleistung durch Tests mit den typischen Benutzern. Bei einem

Test versuchen die Probanden, bestimmte Aufgaben zu lösen, während Beobachter zuschauen, zuhören, Notizen machen und verschiedene Messwerte, wie etwa die benötigte Zeit, aufzeichnen. Solche Tests können potenzielle Probleme und Hindernisse bei der Interaktion mit der Anwendung aufzeigen, qualitative Daten liefern und die Benutzerzufriedenheit überprüfen [38].

Nach der Durchführung von Usability-Tests wurde die zweite Erhebungsmethode, nämlich *Experteninterviews*, eingesetzt, um eine genauere Bewertung der Anwendung durch zusätzliche Perspektiven zu erzielen. Bei dieser Methode geht es darum, spezifisches Wissen und Erkenntnisse von Personen zu gewinnen, die über umfangreiche Erfahrungen und Fachkenntnisse zu einem bestimmten Thema verfügen. Im Rahmen von Interviews werden Experten auf ihrem Gebiet gezielt befragt, um Erkenntnisse, Meinungen und Informationen zu erhalten [39]. Da alle Probanden der Studie aus dem IT-Bereich kamen und das Problem, das die Anwendung lösen sollte, gut verstanden, war es das Ziel, durch die Interviews mit ihnen Einblicke und Erkenntnisse zu gewinnen, die über reine Beobachtungen und Benutzerfeedbacks hinausgehen. Sie hatten die Möglichkeit, ihre Ideen einzubringen, wie die Anwendung benutzerfreundlicher gestaltet werden kann, welche Funktionen fehlen oder welche Änderungen angebracht sind.

6.2 Auswahl von Probanden

Für die Studie wurden Personen mit fundierten IT-Kenntnissen gesucht. Vor allem waren IT-Sicherheitsexperten erwünscht, da sie die eigentliche Zielgruppe darstellen. Die Personen sollten die Grundlagen von Versionskontrollsystemen kennen, d.h. mit Begriffen, wie z.B. „Repository“ oder „Commit“ vertraut sein. Darüber hinaus sollten sie mind. eine Programmiersprache beherrschen und in der Lage sein, den Quellcode anderer Programmiersprachen ungefähr zu verstehen. Dies war wichtig, um den Grund von Änderungen in einer für sie ggf. fremden Programmiersprache zu beurteilen. Eine weitere Voraussetzung war die Beherrschung der englischen Sprache, da sowohl die Anwendung als auch die darin präsentierten Inhalte auf Englisch waren.

Alle Personen, die sich zur Teilnahme bereit erklärten, waren männlich, zwischen 23 - 43 Jahre alt und studierten derzeit bzw. früher Informatik. Die meisten von ihnen waren aus Zeitmangel nicht in der Lage, sich persönlich zu treffen, sodass Videokonferenzen gehalten wurden. Zu Beginn standen Fragen zu ihrer Tätigkeit, ihrer Erfahrung im

Bereich IT-Sicherheit und spezifische Fragen, wie etwa „Welche Sicherheitslücken sind Ihnen bekannt?“, um ihren Wissensstand zu ermitteln und sie auf dieser Basis als Fortgeschrittene oder Anfänger einzustufen. Im Anschluss gab es Fragen zum verwendeten Gerät und Browser, um sowohl den Bedienkomfort auf verschiedenen Geräten als auch die Kompatibilität mit verschiedenen Browsern sicherzustellen. Fragen zum laufenden Betriebssystem und zur Rechnerleistung wurden ausgelassen, da die Anwendung ressourcenintensive Aufgaben auf dem Server ausführt und daher nur wenig von diesen Faktoren beeinflusst wird.

In der folgenden Tabelle sind alle Probanden der Studie mit pseudonymisierten Namen aufgeführt:

Name	**Alter**	**IT-Sicherheitserfahrung**	**Gerät**	**Browser**
Proband 1	30	Fortgeschritten	Desktop	Firefox
Proband 2	43	Anfänger	Desktop	Chrome
Proband 3	30	Anfänger	Laptop	Safari
Proband 4	29	Anfänger	Laptop	Brave
Proband 5	24	Anfänger	Laptop	Firefox
Proband 6	23	Fortgeschritten	Laptop + Maus	Chrome
Proband 7	23	Anfänger	Laptop	Firefox
Proband 8	28	Anfänger	Laptop + Maus	Safari
Proband 9	32	Fortgeschritten	Desktop	Chrome
Proband 10	37	Fortgeschritten	Laptop + Maus	Safari

Tabelle 4: Probanden der Studie

6.3 Experimente

In diesem Abschnitt werden die im Rahmen der Studie durchgeführten Experimente vorgestellt und die Ergebnisse ausgewertet.

6.3.1 Experiment 1

Mit dem ersten Experiment sollten Daten gesammelt werden, um die erste Forschungsfrage zu beantworten. Dabei ging es darum, welche Informationen für Experten bei der Identifikation und Klassifikation von Sicherheitsfixes am relevantesten sind. In diesem Fall war die Anwendung nicht Gegenstand der Forschung, d.h. es wurde kein Usability-Test vorgenommen, sondern sie diente lediglich als Mittel zur Darstellung der zu testenden Informationen.

Vorgehen

Ursprünglich war die Idee, eine bestimmte Anzahl von Commits mit und ohne Sicherheitsfixes auszuwählen, diese nach der Anzahl der Informationsarten zu unterteilen und anschließend die einzelnen Gruppen jeweils mit einer Information zu präsentieren. Die Probanden würden die entsprechende Information nutzen, um den Commit als relevant oder irrelevant einzustufen und ihn in eine CWE-Kategorie einzuordnen. Das Problem lag jedoch in der Verfügbarkeit einiger Informationen. Zum Beispiel sind die CVE-Informationen nicht bei jedem Commit vorhanden, sodass für eine Gruppe nichts angezeigt werden könnte.

Es wurde beschlossen, analog vorzugehen, aber den Probanden alle Informationen über jeden Commit bereitzustellen. Diese Informationen wurden allerdings zu Beginn ausgeblendet und durften nur bei Bedarf einzeln angezeigt werden. Dies sollte verhindern, dass weniger relevante Informationen die Meinung über den Commit beeinflussen. Die Anwendung wurde dabei modifiziert, um die Art der eingeblendeten Informationen und die Reihenfolge zu protokollieren. Auf diese Weise konnte überprüft werden, welche Informationen die Probanden am ehesten benötigen, und ein anschließendes Experteninterview sollte die tatsächliche Nützlichkeit dieser Informationen nachweisen.

Die Zusammenstellung der zu testenden Commits erfolgte selektiv, da einige Commits das Problem mit fehlenden Informationen aufweisen. Es wurden insgesamt nur 15 Commits ausgewählt, um eine Überlastung der Probanden zu vermeiden, da sie die Zuverlässigkeit der Ergebnisse beeinträchtigen könnte. Die Commits stammten jeweils aus einem anderen Repository, um verschiedene Konventionen von Commit-Nachrichten zu berücksichtigen. Je nach Schwachstelle ließen sie sich zusammen in fünf verschiedene Kategorien unterteilen. In jeder Kategorie gab es zwei Sicherheitsfixes, die garantiert Hinweise in den CVE-Informationen oder in der Commit-Nachricht enthielten, und einen vorgetäuschten Sicherheitsfix. Um kontextabhängige Entscheidungen zu vermeiden, wurden die Commits einmal nach dem Zufallsprinzip sortiert. Damit sollten die wichtigsten Fälle bei der tatsächlichen Untersuchung von Commits abgedeckt sein.

Vor dem Experiment wurden die unerfahrenen Probanden über verschiedene Schwachstellen aufgeklärt. Außerdem wurde allen Probanden mitgeteilt, dass sie bei Bedarf auch Informationen aus externen Quellen einbeziehen können.

Ergebnisse

Im Experiment wurden den Probanden die in Tabelle 5 aufgelisteten Commits dargelegt. Die ersten beiden Commits der jeweiligen Gruppe sind Sicherheitsfixes, einer mit und einer ohne CVE, und der dritte ist ein regulärer Commit.

Referenz	Commit	Repository (Kategorie)
1	43157490...	curl (Double-Free)
2	05564e75...	
3	697592b3d...	
4	b5c7491d...	httpd (Denial-of-Service)
5	178a03ca...	
6	5540d05c...	
7	748f478f...	openssl (Out-of-Bounds)
8	428511ca...	
9	4612eec3...	
10	1f863857...	sudo (Buffer-Overflow)
11	10e37223...	
12	0b63af91...	
13	5de6cce5...	systemd (Race-Condition)
14	71161305...	
15	3df566a6...	

Tabelle 5: Für das erste Experiment verwendete Commits aus GitHub-Repositorys

Die Ergebnisse sind in Tabelle 6 abgebildet, wobei die Spalten für die Probanden und die Zeilen für die Commits stehen. Aus den einzelnen Zellen können die eingeblendeten Informationen sowie deren Reihenfolge abgelesen werden. Die roten Zellen bedeuten dabei, dass der jeweilige Proband den Commit falsch einstufte, d.h. einen sicheren Commit als unsicher einschätzte oder umgekehrt, während die gelben Zellen bedeuten, dass der Commit falsch klassifiziert wurde.

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
1	CD	CMD	CMD	CD	CMD	C	CMD	MCD	C	CD
2	CMD	CM	CM	MC	CM	CMD	CM	MD	CM	CM
3	CMD	CM	CMD	CMD	CMD	CMD	CMD	MD	CM	CMD
4	CD	C	C	CD	CD	C	C	M	C	CD
5	CMD	CM	CMD	M	CM	CMD	CM	M	CM	CMD
6	CM	CMD	CM	CMD	CMD	CMD	CMD	MD	CMD	CM
7	CD	C	C	MC	C	CD	C	MC	C	CD
8	CMD	CM	CM	CM	CM	CMD	CM	CM	CM	CM
9	CM	CMD	CM	CMD	CMD	CM	CMD	MD	CM	CM
10	CD	C	C	MC	C	C	C	MC	C	C
11	CMD	CM	CM	CMD	CM	CMD	CM	MD	CMD	CMD
12	CMD	CM	CMD	MD	CMD	CMD	CMD	MD	CMD	CM
13	CD	C	C	C	C	CD	C	MC	CD	CD
14	CMD	CM	CM	CMD	CMD	CMD	CMD	MD	CM	CM
15	CMD	CM	CD	CM	CM	CMD	CMD	MD	CMD	CMD

Tabelle 6: Eingeblendete Informationen für jeden Commit (C - CVE, M - Message, D - Diff, H - History)

Die Tabelle 6 zeigt, dass alle bis auf die Probanden 4 und 8 immer zuerst die CVE-Informationen abriefen. Auf Nachfrage erklärte Proband 4, dass er dies anfangs auch tat, dann aber aufhörte, weil es ihm zu lästig wurde, dass die Informationen oft nicht vorhanden waren. Proband 8 wiederum gab an, dass er sich ggf. unnötige Schritte ersparen wollte und sich deswegen erst die Commit-Nachrichten ansah. Wenn darin ein CVE referenziert wurde, zeigte er die zugehörigen Informationen an.

Obwohl die vorhandenen CVE-Informationen meist für eine korrekte Klassifikation ausreichten, wurden in einigen Fällen weitere Details angesprochen. Zum Beispiel überprüften die Probanden 1 und 9 zusätzlich die Änderungen, um sicherzustellen, dass die im Commit enthaltenen Dateien tatsächlich betroffen sind. Anfänger hingegen sahen sich die Nachricht und ggf. die Änderungen an, wenn sie sich über die Kategorie der Schwachstelle unsicher waren.

Des Weiteren fiel es einigen Probanden schwer, Sicherheitsfixes korrekt einzustufen bzw. zu klassifizieren. Die Mehrzahl der Fehleinstufungen wurde von Anfängern bei vorgetäuschten Sicherheitsfixes getätigt, und sind daher auf die begrenzte Erfahrung dieser Probanden zurückzuführen. Die Fehlklassifikationen hingegen entstanden durch die Unachtsamkeit der Probanden, die offensichtliche Anhaltspunkte in Commits ausließen.

Grundsätzlich äußerten alle Probanden, dass sie die CVE-Informationen am nützlichsten fanden. Anfänger gaben an, dass sie hohe Erwartungen an den Informationswert der Nachrichten hatten und dass diese nur teilweise erfüllt wurden. Erfahrene Probanden bekräftigten diese Aussage, indem sie auf die mangelnde Aussagekraft der Nachrichten hinwiesen. Die Commit-Historie wurde während des gesamten Experiments von niemandem eingesehen. Nach Ansicht der Mehrheit bestand dafür kein Bedarf, da die Commits keinen Hinweis auf eine Verbindung zu anderen Commits gaben.

6.3.2 Experiment 2

Das Ziel des zweiten Experiments war es, herauszufinden, wie benutzerfreundlich die Anwendung im Vergleich zu gängigen Methoden ist.

Vorgehen

Der Vergleichsgegenstand sollte eine Anwendung sein, die standardmäßig zum Betrachten von Commits eingesetzt wird. Die Wahl fiel auf die öffentlich zugängliche UI von GitHub. Sie bietet eine Übersicht über die wichtigsten Informationen zu einem Repository wie den Code, die Issues und mehr. Außerdem ermöglicht sie das Durchsuchen des gesamten Repositorys, die Anzeige von Dateien, die Überprüfung von Commit-Historien sowie den Vergleich von Codeänderungen zwischen Commits. Seit einiger Zeit können auch CVE-Einträge direkt in der UI eingesehen werden. Um den Zugriff zu erleichtern, werden die in Commits angegebenen CVE-IDs automatisch mit dem zugehörigen Eintrag verknüpft.

Sowohl die Anwendung als auch die GitHub-UI wurden jeweils einem Usability-Test unterzogen, bei dem auch der Zeitaufwand gemessen wurde. Beim Test mit GitHub erhielten die Probanden URLs zu drei Sicherheitsfixes, die sie sich mithilfe der verfügbaren Funktionen betrachten sollten. Diese waren nicht besonders komplex, da es nicht darum ging, sie einzuordnen, sondern die UI zu erproben. Anschließend bestand ihre Aufgabe darin, die Nummern der geänderten Zeilen und die Metadaten manuell in eine CSV-Datei zu übertragen. Das strukturierte Datenformat wie CSV würde in der Realität eine einfache Verarbeitung durch Anwendungen und andere Personen gewährleisten. Die Datei sollte die gleichen Felder enthalten wie die Exportdatei, die von der Anwendung erzeugt wird. Hier ist ein Beispiel dafür:

Parent-Hash	Commit-Hash	Dateipfad	Alte Zeilen	Neue Zeilen
8f2079154f80...	6e659993952a...	lib/http.c	730-736,739-741,744-747,749	730-734,737-739,743-745,747

Tabelle 7: Beispiel für das Format, in dem die Probanden die Daten extrahieren sollten

Vor dem Test wurden die Probanden darüber informiert, wie und wo sie die entsprechenden Angaben finden können. Zudem sollten sie die CSV-Datei im Vorfeld anlegen und bereithalten.

Beim Test mit der Anwendung wurde das gleiche Prinzip wie bei der GitHub-UI angewandt. Innerhalb der Anwendung wurde ein Projekt mit 10 Commits vorbereitet, das sowohl Sicherheitsfixes als auch reguläre Commits enthielt. Die Probanden sollten diese Commits mit den verfügbaren Funktionen untersuchen und bewerten, ohne sich allzu viel Mühe zu geben. Die Anzahl der Commits wurde dabei höher gewählt, um sicherzustellen, dass mögliche Probleme der Benutzerfreundlichkeit, wie z.B. Schwierigkeiten oder Verzögerungen bei der Interaktion, nicht übersehen werden. Am Ende der Bewertung sollten die Probanden eine Exportdatei erstellen lassen.

Ergebnisse

Um den Vergleich zwischen der GitHub-UI und der implementierten Anwendung besser nachvollziehen zu können, wird im Folgenden ein Bild der Commit-Detailansicht für jede der beiden Anwendungen gezeigt:

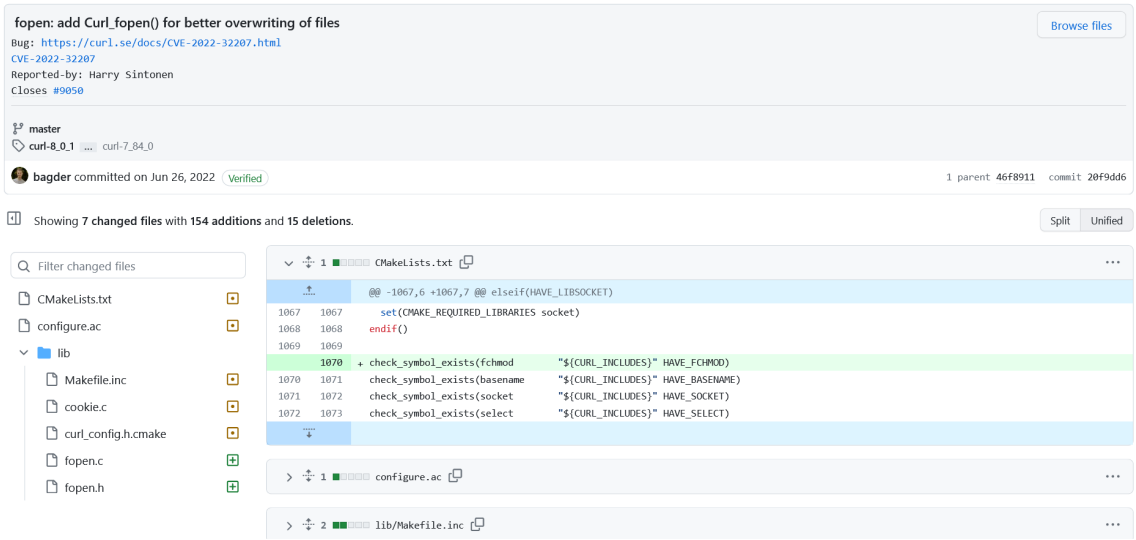

Abbildung 7: Commit-Ansicht in der GitHub-UI

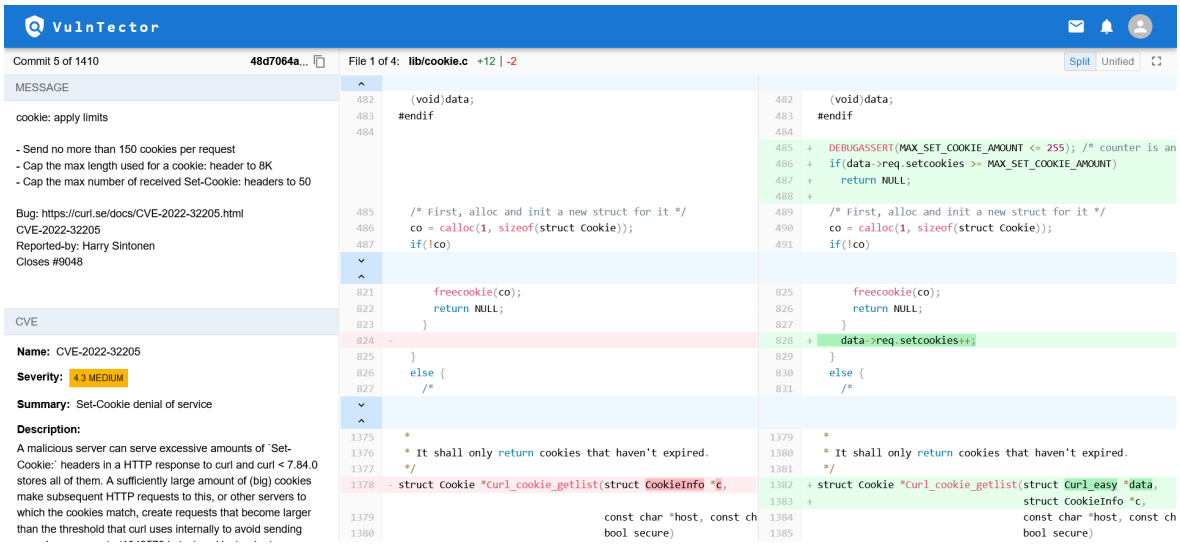

Abbildung 8: Commit-Ansicht in der implementierten Anwendung

Der wichtigste Aspekt, der allen zuerst auffiel, war die Gestaltung der implementierten Anwendung. Die Probanden äußerten sich positiv darüber, wie die Informationen dargestellt werden. Sie gaben an, dass wichtige Informationen, insbesondere zu CVE, auf einen Blick verfügbar sind. Die Probanden 2, 6 und 8 wünschten sich zusätzlich, die Namen aller betroffenen Dateien zu sehen. Alle drei waren sich einig, dass ein Seitenfenster wie in GitHub-UI, das zudem per Hotkey ein- und ausgefahren werden kann, nützlich wäre, um den Überblick zu behalten und die Möglichkeit zu haben, direkt zu einer bestimmten Datei zu wechseln.

Proband 6 fand es auch in Ordnung, wie die GitHub-UI alle betroffenen Dateien eines Commits in einer Ansicht darstellt. Dadurch könne man Änderungen in verschiedenen Dateien schnell abgleichen, um den Zusammenhang zu verstehen.

Den Probanden 5 und 6 gefiel es nicht, dass in Commit-Nachrichten die URLs zu weiterführenden Ressourcen nicht verlinkt werden. Proband 5 (Anfänger) machte sich die Mühe, die URLs manuell zu kopieren und in einem neuen Tab zu öffnen. Er und Proband 8 wünschten sich auch, dass Verweise auf Issues verlinkt werden, sodass man direkt zur Diskussion über das jeweilige Problem gelangen kann.

Die Probanden 1, 3, 5 und 10 brauchten mehr Informationen über die angezeigten CVE-Klassen. Sie wollten die CVSS-Metriken sehen, um den Schweregrad bewerten zu können, und fanden es praktisch, wie sie in der GitHub-UI dargestellt werden. Zudem wollten sie

die CWE-Informationen sehen, aber unmittelbar, ohne erst mehrere Zwischenschritte bewältigen zu müssen, wie es bei der UI der Fall ist. Diese Informationen würden die Entdeckung von Schwachstellen im Quellcode sowie deren Klassifikation erleichtern, betonten sie im Hinblick auf die Erfahrungen aus dem ersten Experiment. Nach Ansicht der Probanden 1 und 3 wäre dabei die Anzeige von Beispielcodes für die Schwachstellen besonders hilfreich. Als alternative Lösung schlug Proband 5 vor, eine URL zum kompletten CVE-Eintrag beizufügen.

Alle Probanden gaben an, dass die Darstellung von Commit-Änderungen sowohl in der GitHub-UI als auch in der Anwendung anschaulich ist. Proband 1 wünschte sich noch die Möglichkeit, die Definition der aufgerufenen Funktionen im Quellcode anzuzeigen und dahin zu navigieren, wie es in der UI vorgesehen ist. Dies würde ein besseres Verständnis des Quellcodes und seiner Beziehung zu den Funktionen ermöglichen. Das folgende Beispiel zeigt, wie dies in der UI realisiert ist:

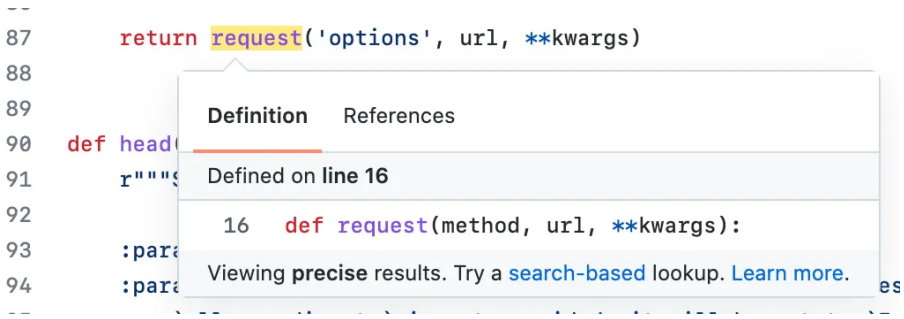

Abbildung 9: Anzeige der Definition einer Funktion in der GitHub-UI [40]

Was Probanden 1 und 7 noch gefiel, ist die Möglichkeit, Commit-Historie übersichtlich auszugeben, auch wenn sie dies nur zu Testzwecken nutzten. Proband 1 wünschte sich zudem, dass auch nachfolgende Commits angezeigt werden und dass er über die Historie zu einem Commit navigieren kann.

Die Probanden 5 und 9 vermissten den dunklen Modus, der aber in der GitHub-UI vorhanden ist. Sie erklärten, dass sie in der Regel den hellen Modus vermeiden, weil er ihre Augen bei längerer Nutzung anstrengt und ermüdet.

Ergebnisse - Zeitvergleich

Die Zeitmessungen während der Usability-Tests ergaben folgende Werte (in Minuten):

Proband	GitHub-UI (3 Commits)	Impl. Anwendung (10 Commits)
P1	06:05	01:02
P2	06:41	01:33
P3	08:03	01:28
P4	07:58	01:49
P5	07:46	02:19
P6	06:28	01:51
P7	07:21	02:04
P8	07:13	02:06
P9	06:39	01:24
P10	06:17	01:45

Tabelle 8: Gemessene Zeiten während der Usability-Tests

Zunächst fällt auf, dass die Zeiten für die Aufgabenlösung mit der GitHub-UI und der implementierten Anwendung deutlich voneinander abweichen. Im Durchschnitt dauert die manuelle Datenextraktion über die UI etwa 2 Minuten und 42 Sekunden pro Commit, während es mit der Anwendung nur 17 Sekunden sind. Dies entspricht einer Reduzierung um den Faktor 14, was auf eine hohe Produktivität bei der Nutzung der Anwendung schließen lässt.

Bemerkenswert ist auch, dass die Zeiten bei der Verwendung der GitHub-UI erheblich variieren und bei Laptop-Nutzern etwas höher liegen. Im Gegensatz dazu sind die Zeitschwankungen bei der Anwendung geringer, und sowohl Desktop- als auch Laptop-Benutzer sind in der Lage, die Aufgabe in einer ähnlich kurzen Zeit zu erledigen. Das bedeutet, dass die Anwendung eine konsistente Arbeitsgeschwindigkeit bietet, unabhängig von der Art des verwendeten Geräts. Dabei könnten insbesondere die Hotkeys eine entscheidende Rolle spielen, da sie eine einheitliche Arbeitsweise für alle Benutzertypen ermöglichen.

Des Weiteren zeigten die Beobachtungen eine hohe Fehleranfälligkeit bei der manuellen Datenextraktion. Während der Übernahme von Zeilennummern unterliefen den Probanden 4 und 7 mehrere Flüchtigkeitsfehler, die sie nur teilweise selbständig erkennen und korrigieren konnten. Abgesehen davon, dass die Korrektur Zeit kostet, könnten solche unerkannten Fehler in der Realität zu größeren Problemen führen, die zudem schwer zu entdecken wären. Die von der Anwendung gebotene Automatisierung schließt dies jedoch aus und garantiert somit eine hohe Effektivität.

6.3.3 Experiment 3

Das Ziel des dritten Experiments war die Evaluation der Anwendung als Ganzes. Dabei sollten die Probanden eine umfassende Bewertung aller Aspekte, Funktionen und Leistungen der Anwendung vornehmen, um ihre Gesamtqualität zu beurteilen.

Vorgehen

Um den Probanden eine grundlegende Einführung in die gesamte Anwendung zu geben, sollten sie zu Beginn verschiedene Aufgaben mit der Anwendung erledigen, z.B. ein Projekt erstellen oder eine Einladung versenden. Dies gab ihnen die Möglichkeit, die wichtigsten Funktionen außerhalb der Commit-Ansicht kennenzulernen. Anschließend konnten sie frei in der Anwendung navigieren, und alle verfügbaren Funktionen und grafischen Teile näher betrachten bzw. austesten.

Weiterhin erhielten die Probanden je ein *SUS*-Fragebogen (*System Usability Scale*), den sie anhand ihrer gemachten Erfahrungen ausfüllen sollten. Es handelt sich dabei um einen standardisierten Fragebogen zur Bewertung der Benutzerfreundlichkeit von Software, Websites und anderen technischen Produkten [41]. In qualitativen Forschungen wird er normalerweise nicht verwendet, da er für geschlossene Fragen konzipiert ist, bei denen die Teilnehmer aus vordefinierten Antwortoptionen wählen. In diesem Fall wurde er jedoch als Ergänzung zu den qualitativen Daten verwendet, um einen möglichst umfassenden Einblick in die Benutzererfahrung zu erhalten.

Ergebnisse

Hinsichtlich der Aufgabenangemessenheit waren alle Probanden darin einig, dass sie mit der Anwendung die gestellten Aufgaben effizient bzw. mit wenig Aufwand erledigen konnten. Was jedoch laut Proband 7 noch fehlte, ist die Hervorhebung der Schwachstellenbegriffe in Angaben wie Commit-Nachricht und CVE-Beschreibung. Dies würde sicherstellen, dass ein aufschlussreicher Sicherheitsfix sofort erkannt und klassifiziert wird. Die Beobachtungen bestätigten die Notwendigkeit dieser Funktion. Die meisten Probanden überprüften die Angaben zuerst auf den Namen einer Schwachstelle und verbrachten dabei viel Zeit mit langen Inhalten. Ferner wurden angesprochene Schwachstellen von einigen Anfängern überlesen, was dann zu einer falschen Einstufung bzw. Klassifikation führte. Nach Ansicht der Probanden 1 und 9 könnte eine andere

Verbesserung die Möglichkeit sein, zwischen einzelnen Zeilenänderungen umzuschalten, um mühsames Scrollen bei der ausführlichen Anzeige von Dateiinhalten zu vermeiden.

Was die Selbstbeschreibungsfähigkeit der Anwendung angeht, war die Mehrheit mit dem aktuellen Stand zufrieden. Von den übrigen Probanden gab es kleinere Verbesserungsvorschläge. Die Probanden 2 und 5 wünschten sich mehr Feedback von der Anwendung zu erhalten. Zum Beispiel könnte nach der Abgabe einer Bewertung eine entsprechende Bestätigungsnachricht erscheinen. Dadurch würde man sich vergewissern, dass ihm kein Fehler unterlaufen ist und dass die Bewertung erfolgreich registriert wurde. Die Probanden 4 und 5 bemerkten zudem, dass es keine Tooltips für die Buttons gibt, sodass man nicht genau weiß, welche Funktion mit ihnen verbunden ist.

Allen Aussagen zufolge wurde die Erwartungskonformität erfüllt. Die einzige Anmerkung diesbezüglich kam von Proband 2, der bemängelte, dass es keine offensichtlichen Navigationsbuttons gibt, um zur vorherigen Seite bzw. zur Hauptseite zu gelangen. Dieser Mangel wurde auch in den Beobachtungen deutlich: Einige Probanden suchten beim Zurückgehen nach diesen Buttons und verschwendeten dadurch Zeit.

Die Anwendung war auch für das Lernen ausreichend förderlich. Die meisten Probanden äußerten sich positiv über die Anzeige und Erklärung der vorhandenen Hotkeys. Den Probanden 4 und 7 gefiel besonders gut, dass man die Hilfe auch mit Hotkey ein- und ausblenden kann und somit nicht gezwungen ist, auf die Maus zurückzugreifen. Als sinnvolle Ergänzung schlug Proband 1 vor, Erstnutzern ein interaktives Tutorial anzubieten, das ihnen die wichtigsten Funktionen der Anwendung näher bringt. Dies würde eine schnellere und effektivere Einarbeitung ermöglichen und jegliche Unsicherheiten oder Fragen minimieren.

Die Steuerbarkeit der Anwendung war für die Probanden angemessen. Der einzige Wunsch wurde von den Probanden 2 und 9 geäußert, und betraf die Möglichkeit, abgegebene Bewertungen zurückzunehmen, um zu vermeiden, dass versehentlich bewertete irrelevante Dateien in die Exportdatei aufgenommen werden.

Aus den Testbeobachtungen ließ sich schließen, dass die Fehlertoleranz hoch ist. Alle Funktionen arbeiteten wie vorgesehen, Abbrüche wurden ordnungsgemäß gehandhabt und es konnten keine Auffälligkeiten festgestellt werden. Die Probanden 7 und 9 machten

absichtlich falsche Eingaben in verschiedenen Bereichen, wie z.B. im Dialogfenster für die Projekterstellung, aber es zeigten sich keine negativen Auswirkungen.

Was die Individualisierbarkeit anbelangt, so standen den Probanden nur begrenzte Optionen zur Verfügung. Obwohl sich keiner von ihnen darüber beschwerte, wurden einige Optionen genannt, die als Ergänzung wünschenswert wären. Proband 6 erwähnte die Möglichkeit, die Größe der einzelnen Informationsfenster zu verändern. Dadurch könnten wenig relevante Fenster verkleinert werden, um mehr Platz für die Darstellung der anderen Informationen zu schaffen und sich besser auf diese zu konzentrieren. Weiterhin solle es möglich sein, die Fenster nach eigenen Präferenzen auszurichten und die Einstellung zu speichern, wie es in den meisten IDEs (Integrated Development Environments) üblich ist. Insgesamt würde diese Flexibilität auch den Entwickler von der Sorge befreien, wie die Darstellung am besten zu gestalten ist.

Ergebnisse - Fragebogen

Der SUS-Fragebogen besteht aus 10 Fragen, die sowohl positive als auch negative Aussagen enthalten, um eine ausgewogene Bewertung zu ermöglichen. Die Teilnehmer bewerten ihre Zustimmung zu den Aussagen auf einer Skala von 0 (stimme überhaupt nicht zu) bis 4 (stimme voll und ganz zu) [41]. Die Aussagen sind wie folgt formuliert:

1. Ich denke, dass ich das System häufig regelmäßig nutzen würde.
2. Das System erschien mir unnötig komplex.
3. Ich fand das System einfach zu bedienen.
4. Ich glaube, ich bräuchte technische Unterstützung, um das System bedienen zu können.
5. Ich fand, dass die verschiedenen Funktionen im System gut integriert sind.
6. Ich fand, dass das System zu uneinheitlich war.
7. Ich könnte mir vorstellen, dass die meisten Menschen die Nutzung des Systems sehr schnell erlernen würden.
8. Ich fand das System sehr umständlich zu bedienen.
9. Ich fühlte mich sehr sicher beim Arbeiten mit dem System.
10. Ich musste eine Menge erfahren, bevor ich das System anwenden konnte [41].

Die Ergebnisse des Fragebogens sehen wie folgt aus:

Proband	**Antworten**										**Score (0-100)**
	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	
1	4	0	4	3	4	0	3	0	3	1	85.0
2	4	0	3	3	4	1	3	1	3	2	75.0
3	4	0	4	2	4	0	4	0	3	1	90.0
4	4	0	4	2	4	1	3	1	3	2	80.0
5	4	0	4	2	4	1	3	0	3	2	82.5
6	4	0	4	2	3	0	3	0	4	2	85.0
7	4	0	3	2	4	0	4	0	3	0	90.0
8	4	0	4	2	4	0	3	0	3	1	87.5
9	4	0	4	3	4	0	4	0	3	1	87.5
10	4	0	4	2	4	0	4	0	3	1	90.0

Tabelle 9: Ergebnisse des SUS-Fragebogens

Die Ergebnisse zeigen noch einmal deutlich, dass die Anwendung Schwächen bei der Vermittlung des Sicherheitsgefühls hat. Wie bereits erkannt, ist dies auf das fehlende Feedback sowie teilweise mangelnde Beschreibung zurückzuführen. Darüber hinaus ist technische Unterstützung vonnöten, um sich mit der Anwendung vertraut zu machen. Insgesamt ist die Benutzerfreundlichkeit jedoch auf einem hohen Niveau, was durch den SUS-Wert von durchschnittlich 85,25 bestätigt wird. Die einzelnen Scores wurden dabei anhand der folgenden Formel berechnet:

```
SUS Score = (
    (Q1−1) + (Q3−1) + (Q5−1) + (Q7−1) + (Q9 − 1) +
    (5 − Q2) + (5 − Q4) + (5 − Q6) + (5 − Q8) + (5 − Q10)
) * 2.5
```

Abbildung 10: Formel zur Berechnung des SUS-Scores

6.4 Diskussion der Ergebnisse

In diesem Abschnitt werden zunächst die Ergebnisse der Studie zusammengefasst. Dabei wird versucht, die gestellten Forschungsfragen zu beantworten. Im Anschluss daran werden Faktoren, die die Studie gefährden konnten, sowie Einschränkungen aufgeführt.

6.4.1 Interpretation

Informationen

In Bezug auf die erste Forschungsfrage, „Welche Informationen können die Identifikation und Klassifikation von Sicherheitsfixes verbessern?“, stellte sich heraus, dass die CVE-Informationen sowohl für Anfänger als auch für Experten am relevantesten sind. Wenn sie nicht zur Verfügung stehen, werden die Commit-Nachrichten vorgezogen. Der Aussagewert dieser Nachrichten kann aber je nach Autor und Konventionen im jeweiligen Softwareprojekt stark variieren. In einigen Fällen können sie Aufschluss über die Kategorie des Sicherheitsproblems geben, und in anderen Fällen ist es nicht einmal möglich zu entscheiden, ob ein Sicherheitsproblem vorliegt. In diesem Fall wird auf die Commit-Änderungen zurückgegriffen, um sie auf bekannte Schwachstellenmuster zu untersuchen. Allerdings ist dies nicht ganz einfach, da sich die Struktur des betroffenen Codes als komplex darstellen kann. Außerdem werden die Änderungen auch bei Unsicherheiten überprüft oder um sicherzustellen, dass die Beschreibung und der Quellcode übereinstimmen. Information wie die Commit-Historie wies eine niderige Relevanz auf, was jedoch durch die ausgewählten Commits begründet sein könnte (siehe Abschnitt 6.4.2).

Anwendung

Was die zweite Forschungsfrage betrifft, so wurde festgestellt, dass die Tastenbedienung besonders nützlich ist. Selbst Personen, die hauptsächlich eine Maus benutzen, gaben an, dass sie sich schnell an diese Bedienung gewöhnten und dass sie damit ihre Arbeit effizienter gestalten konnten. Dies gilt insbesondere auch für Benutzer ohne Maus. Ferner ergab sich, dass wichtige Informationen unmittelbar zur Verfügung stehen müssen. Die Suche nach ihnen kostet Zeit und Energie, was die Arbeit spürbar behindern kann. Weiterhin zeigte sich, dass eine schnelle und effiziente Navigation zwischen den Commits eine große Rolle spielt, wenn eine Mehrzahl von Commits vorliegt. So konnten im zweiten

Experiment deutlich niedrigere Zeiten erreicht werden, was wiederum die Produktivität erhöhte. Dann wurde noch deutlich, dass eine Anwendung die direkte Konfrontation mit den später zu verarbeitenden Daten aufheben sollte. Denn es zeigt sich, dass die manuelle Übertragung von Daten die Fehleranfälligkeit erhöht, was wiederum kostspielige Folgen haben kann. Schließlich ergab sich, dass eine Anwendung Sicherheit und Verständlichkeit gewährleisten sollte, da dies sonst die Benutzer daran hindert, die Anwendung schnell zu erlernen und sie effizient einzusetzen.

6.4.2 Validität

Der erste Faktor, der die Ergebnisse sowohl im ersten als auch geringfügig im zweiten Experiment beeinflusst haben könnte, war die Erfahrung der Probanden. Im ersten Fall bestand das Ziel darin, den Sicherheitsstatus der Commits zu bestimmen und ggf. zu klassifizieren. Dies wurde ohne Fehler hauptsächlich von den Sicherheitsforschern erledigt. Sie waren umfassend über Schwachstellen und deren Muster informiert, und Proband 1 war sogar unmittelbar mit dem Repository-Mining zu Sicherheitszwecken vertraut. Die Anfänger hingegen verfügten nur über ein Grundwissen, das durch eine Einführung vor dem Experiment aufgefrischt wurde, aber möglicherweise nicht ausreichte, um ihre Ergebnisse tatsächlich zu würdigen. Zudem könnten auch Programmiersprachkenntnisse bei komplexen Codeänderungen eine Rolle gespielt haben.

Ein weiterer Einflussfaktor könnte die Komplexität der im ersten Experiment verwendeten Commits sein. Diese wurden so ausgewählt, dass sie von Anfängern und Experten gleichermaßen verstanden werden können. Letztlich ist aber eine solche Auswahl subjektiv, sodass die Commits, die dem Selektor angemessen erschienen, für einige Probanden komplex und für andere zu einfach gewesen sein konnten.

Weiterhin gab es eine begrenzte Anzahl und Vielfalt der verwendeten Commits. Zahlreiche Schwachstellen und Programmiersprachen wurden nicht berücksichtigt. Dies könnte dazu geführt haben, dass realistische Szenarien nicht ausreichend abgebildet wurden, um allgemeine Schlussfolgerungen zu ziehen.

6.4.3 Limitierungen

Die erste Einschränkung war die geringe Anzahl von Probanden aus der eigentlichen Zielgruppe. Aus Zeitgründen konnten nicht genügend von Sicherheitsexperten erreicht werden, um eine möglichst aussagekräftige Studie durchzuführen. Zusätzlich stützte sich die Untersuchung auf begrenzte Testdaten. Aus Zeiteinschränkungen konnten nur wenige Commits getestet werden, sodass zahlreiche Fälle unberücksichtigt blieben. Schließlich konnten einige Funktionen in der Anwendung nicht implementiert werden, sodass nicht alle Informationen und Funktionen überprüft werden konnten. Der Nutzwert der fehlenden Elemente kann daher nicht eindeutig beurteilt werden.

7 Fazit und Ausblick

Fazit

Zu Beginn dieser Arbeit wurde die aktuelle Situation auf dem Gebiet der statischen Analyse betrachtet. Es stellte sich heraus, dass derzeit ML-Verfahren eruiert werden, um ihren Gebrauchswert für die Analyse zu ermitteln. Dazu sind allerdings große Mengen an hochwertigen Datensätzen erforderlich, die nur schwer zu beschaffen sind. Diese Schwierigkeiten sind schlecht zugängliche Informationen, geringe Effizienz aufgrund des Fehlens einer geeigneten Umgebung und der Kollaborationsmöglichkeit. Daher wurde entschieden, diese Probleme durch die Bereitstellung einer Anwendung zu beseitigen, die den Prozess weitestgehend automatisieren sollte. Dabei tauchten zwei wichtige Fragen auf, nämlich welche Funktionen eine solche Anwendung haben muss und welche Informationen helfen würden, sicherheitskritische Commits schnellstmöglich zu identifizieren. Die Beantwortung dieser Fragen wurde als Ziel der Arbeit gesetzt.

Nach der Implementierung wurde die Anwendung im Rahmen einer qualitativen Studie evaluiert. Dabei ergaben sich die folgenden wichtigen Erkenntnisse:

1. Die Arbeit mithilfe der Anwendung ist viel effizienter als mit herkömmlichen Methoden, wobei auch die Produktivität und Effektivität deutlich höher liegen.

2. Die CVE-Informationen sind für die Identifikation von sicherheitskritischen Commits besonders wichtig.

Ausblick

Aus der Evaluation konnten wertvolle Erkenntnisse abgeleitet werden, die in Zukunft umgesetzt und erprobt werden könnten. Die Tatsache, dass CVE-Informationen besonders relevant sind, könnte genutzt werden, um gezielt nach CVEs in Commits zu suchen, anstatt nach Commits mit CVEs zu suchen. So gibt es in der GitHub-UI rudimentäre Funktionen, um Repositorys nach Schlüsselwörtern zu durchsuchen.

Im Hinblick auf andere Informationen hieß es, dass CWE-Informationen nützlich wären, um die Untersuchung von Commit-Änderungen zu erleichtern und eine sofortige Klassifikation zu ermöglichen. Außerdem könnten CVSS-Metriken dabei helfen, den Schweregrad der Sicherheitslücke zu bestimmen, während die in Commits referenzierten Issues ebenfalls bedeutende Informationen liefern könnten.

Des Weiteren wurden mehrere Verbesserungsvorschläge für die Anwendung eingebracht. Einer der Vorschläge war die Ausgabe der im Commit enthaltenen Dateien, um sowohl den Überblick zu behalten als auch schnell zu einer anderen Datei wechseln zu können. Dann gab es noch die Anregung, die in Nachrichten angegebenen Referenzen zu verlinken, um einen direkten Aufruf der entsprechenden Informationen zu ermöglichen. In den Nachrichten sowie in den CVE-Beschreibungen genannte Schwachstellen sollten hervorgehoben werden, um ihre Identifikation zu optimieren. Zum Verständnis der Zusammenhänge sollte es noch möglich sein, in den Quellcodes über verschiedene Funktionen zu navigieren. Schließlich sollte die Anwendung so anpassbar sein, dass die Fenster mit den Informationen nach Belieben manipuliert werden können.

Literatur

[1] B. Chess und J. West, *Secure Programming with Static Analysis*. Pearson Education, 29. Juni 2007, 1168 S., Google-Books-ID: GL8AeTCu1WAC, ISBN: 978-0-13-270202-7.

[2] R. T. Fielding, „Architectural styles and the design of network-based software architectures," Publication, University of California, Irvine, 2000. Adresse: `https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm` (besucht am 17. 08. 2022).

[3] V. Bojinov, *RESTful Web API Design with Node.js*. Packt Publishing Ltd, 26. Mai 2016, 144 S., Google-Books-ID: cdJtDQAAQBAJ, ISBN: 978-1-78646-320-3.

[4] D. Hardt, „The OAuth 2.0 Authorization Framework," Internet Engineering Task Force, Request for Comments RFC 6749, Okt. 2012, Num Pages: 76. DOI: `10.17487/RFC6749`. Adresse: `https://datatracker.ietf.org/doc/rfc6749` (besucht am 22. 08. 2022).

[5] M. Jones, B. Campbell und C. Mortimore, „JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants," Internet Engineering Task Force, Request for Comments RFC 7523, Mai 2015, Num Pages: 12. DOI: `10.17487/RFC7523`. Adresse: `https://datatracker.ietf.org/doc/rfc7523` (besucht am 02. 09. 2022).

[6] o.V. „Getting Started – React." (4. Feb. 2023), Adresse: `https://reactjs.org/docs/getting-started.html` (besucht am 04. 02. 2023).

[7] Codecademy Team. „React: The virtual DOM," Codecademy. (21. Mai 2023), Adresse: `https://www.codecademy.com/article/react-virtual-dom` (besucht am 21. 05. 2023).

[8] o.V. „Overview | CVE." (20. Apr. 2023), Adresse: `https://www.cve.org/About/Overview` (besucht am 20. 04. 2023).

[9] o.V. „CNA Rules | CVE." (21. Mai 2023), Adresse: `https://www.cve.org/ResourcesSupport/AllResources/CNARules#section_8-1_cve_record_information_requirements` (besucht am 21. 05. 2023).

[10] Daniel Gustafsson. „smtp: avoid risk of buffer overflow in strtol · curl/-curl@39df407." (4. Feb. 2019), Adresse: `https://github.com/curl/curl/commit/39df4073e5413fcdbb5a38da0c1ce6f1c0ceb484` (besucht am 21. 03. 2023).

[11] o.V. „CWE - About - CWE Overview.“ (22. Nov. 2022), Adresse: https://cwe.mitre.org/about/index.html (besucht am 07. 05. 2023).

[12] CWE Content Team. „CWE - CWE-121: Stack-based Buffer Overflow (4.11).“ (27. Apr. 2023), Adresse: https://cwe.mitre.org/data/definitions/121.html (besucht am 07. 05. 2023).

[13] K. Neath. „Issues 2.0: The next generation,“ The GitHub Blog. (10. Apr. 2011), Adresse: https://github.blog/2011-04-09-issues-2-0-the-next-generation/ (besucht am 07. 05. 2023).

[14] A. C. Chavalier, J. P. S. Alcocer und A. Bergel, „Assessing textual source code comparison: Split or unified?“ In *Conference Companion of the 4th International Conference on Art, Science, and Engineering of Programming*, Porto Portugal: ACM, 23. März 2020, S. 50–54, ISBN: 978-1-4503-7507-8. DOI: 10.1145/3397537.3398471. Adresse: https://dl.acm.org/doi/10.1145/3397537.3398471 (besucht am 08. 05. 2023).

[15] J. Cardello. „Web Carousel Design 101: Ideas, Examples, Best Practices.“ (16. Aug. 2022), Adresse: https://www.vev.design/blog/web-carousel-design/ (besucht am 21. 05. 2023).

[16] T. Bray, „The JavaScript Object Notation (JSON) Data Interchange Format,“ Internet Engineering Task Force, Request for Comments RFC 8259, Dez. 2017, Num Pages: 16. DOI: 10.17487/RFC8259. Adresse: https://datatracker.ietf.org/doc/rfc8259 (besucht am 08. 05. 2023).

[17] J. Kari, „Rice's theorem for the limit sets of cellular automata,“ *Theoretical Computer Science*, Jg. 127, Nr. 2, S. 229–254, 23. Mai 1994, ISSN: 0304-3975. DOI: 10.1016/0304-3975(94)90041-8. Adresse: https://www.sciencedirect.com/science/article/pii/0304397594900418 (besucht am 08. 05. 2023).

[18] K. E. Wiegers und J. Beatty, *Software Requirements*. Microsoft Press, 2013, 637 S., Google-Books-ID: 40lDmAEACAAJ, ISBN: 978-0-7356-7966-5.

[19] R. E. Silverman, *Git Pocket Guide: A Working Introduction*. Ö'Reilly Media, Inc.", 25. Juni 2013, 289 S., Google-Books-ID: tPv2vX2czJoC, ISBN: 978-1-4493-2752-1.

[20] M. Ray. „Clustered and nonclustered indexes described - SQL server.“ (3. März 2023), Adresse: https://learn.microsoft.com/en-us/sql/relational-databases/indexes/clustered-and-nonclustered-indexes-described (besucht am 06. 04. 2023).

[21] M. Adler und J.-l. Gailly. „zlib 1.2.13 Manual.“ (13. Okt. 2020), Adresse: https://www.zlib.net/manual.html (besucht am 19. 01. 2023).

[22] o.V. „Same-origin policy - web security | MDN.“ (26. Feb. 2023), Adresse: https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy (besucht am 09. 05. 2023).

[23] World Wide Web Consortium (W3C). „Fetch Standard.“ (8. Mai 2023), Adresse: https://fetch.spec.whatwg.org/ (besucht am 09. 05. 2023).

[24] L. Bass, P. Clements und R. Kazman, *Software Architecture in Practice: Software Architect Practice_ c3*. Addison-Wesley, 25. Sep. 2012, 859 S., ISBN: 978-0-13-294278-2.

[25] o.V. „General Python FAQ,“ Python documentation. (21. Mai 2023), Adresse: https://docs.python.org/3/faq/general.html (besucht am 21. 05. 2023).

[26] S. Zunke und V. D'Souza, „JSON vs XML: A comparative performance analysis of data exchange formats,“ Jg. 3, Nr. 4, 2014. Adresse: http://ijcsn.org/IJCSN-2014/3-4/JSON-vs-XML-A-Comparative-Performance-Analysis-of-Data-Exchange-Formats.pdf (besucht am 15. 02. 2023).

[27] o.V. „Welcome to Flask — Flask Documentation (2.3.x).“ (21. Mai 2023), Adresse: https://flask.palletsprojects.com/en/2.3.x/#user-s-guide (besucht am 21. 05. 2023).

[28] o.V. „About SQLite.“ (21. Mai 2023), Adresse: https://www.sqlite.org/about.html (besucht am 21. 05. 2023).

[29] o.V. „Overview - material UI.“ (21. Mai 2023), Adresse: https://mui.com/material-ui/getting-started/overview/ (besucht am 21. 05. 2023).

[30] R. Gabriel und H.-P. Röhrs, *Datenbanksysteme: Konzeptionelle Datenmodellierung und Datenbankarchitekturen* (Springer-Lehrbuch). Berlin, Heidelberg: Springer, 1995, ISBN: 978-3-540-60079-4 978-3-642-57747-5. DOI: 10.1007/978-3-642-57747-5. Adresse: http://link.springer.com/10.1007/978-3-642-57747-5 (besucht am 21. 05. 2023).

[31] Brumm, Ben. „Database keys: The complete guide (surrogate, natural, composite & more),“ Database Star. (10. Juli 2018), Adresse: https://www.databasestar.com/database-keys/ (besucht am 21. 05. 2023).

[32] M. Ekuan. „Data partitioning guidance - azure architecture center.“ (20. Jan. 2023), Adresse: https://learn.microsoft.com/en-us/azure/architecture/best-practices/data-partitioning (besucht am 20. 01. 2023).

[33] o.V., „DIN EN ISO 9241-110:2020-10, Ergonomie der Mensch-System-Interaktion_- Teil_110: Interaktionsprinzipien (ISO_9241-110:2020); Deutsche Fassung EN_ISO_9241-110:2020," Beuth Verlag GmbH, Okt. 2020. DOI: 10.31030/3147467. Adresse: https://www.beuth.de/de/-/-/320862700 (besucht am 16. 05. 2023).

[34] M. Broy und M. Kuhrmann, „Vorgehensmodelle in der Softwareentwicklung," in *Einführung in die Softwaretechnik*, Ser. Xpert.press, M. Broy und M. Kuhrmann, Hrsg., Berlin, Heidelberg: Springer, 2021, S. 83–124, ISBN: 978-3-662-50263-1. DOI: 10.1007/978-3-662-50263-1_3. Adresse: https://doi.org/10.1007/978-3-662-50263-1_3 (besucht am 10. 05. 2023).

[35] M. Moreno Haller. „Was versteht man unter einem Softwareentwicklungsmodell?" (22. Sep. 2013), Adresse: https://docplayer.org/9644841-Was-versteht-man-unter-einem-softwareentwicklungsmodell.html (besucht am 10. 05. 2023).

[36] N. Döring und J. Bortz, *Forschungsmethoden und Evaluation in den Sozial- und Humanwissenschaften* (Springer-Lehrbuch). Berlin, Heidelberg: Springer, 2016, ISBN: 978-3-642-41088-8 978-3-642-41089-5. DOI: 10.1007/978-3-642-41089-5. Adresse: https://link.springer.com/10.1007/978-3-642-41089-5 (besucht am 11. 05. 2023).

[37] S. Peters. „Qualitative und quantitative Usability-Studien: Wie viele Testpersonen wirklich notwendig sind," Usabilityblog.de. (27. Okt. 2021), Adresse: https://www.usabilityblog.de/qualitative-und-quantitative-usability-studien-wie-viele-testpersonen-wirklich-notwendig-sind/ (besucht am 22. 05. 2023).

[38] A. S. f. P. Affairs. „Usability testing." Publisher: Department of Health and Human Services. (13. Nov. 2013), Adresse: https://www.usability.gov/how-to-and-tools/methods/usability-testing.html (besucht am 11. 05. 2023).

[39] A. Bogner, B. Littig und W. Menz, Hrsg., *Das Experteninterview*, Wiesbaden: VS Verlag für Sozialwissenschaften, 2002, ISBN: 978-3-8100-3200-3 978-3-322-93270-9. DOI: 10.1007/978-3-322-93270-9. Adresse: http://link.springer.com/10.1007/978-3-322-93270-9 (besucht am 11. 05. 2023).

[40] o.V. „Navigating code on GitHub," GitHub Docs. (18. Mai 2023), Adresse: https://docs.github.com/en/repositories/working-with-files/using-files/navigating-code-on-github (besucht am 18. 05. 2023).

[41] A. S. f. P. Affairs. „System usability scale (SUS).“ Publisher: Department of Health and Human Services. (6. Sep. 2013), Adresse: https://www.usability.gov/how-to-and-tools/methods/system-usability-scale.html (besucht am 21. 05. 2023).

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Bremerhaven, den 7. Juni 2023

Adam Cihasev